
FROM EXPERIENCE TO EXPERTISE: ADOPTION-AWARE MEMORY LEARNING FOR DATA-SCARCE NPU KERNEL SYNTHESIS

Longxiao Fan

Fudan University
24110240020@m.fudan.edu.cn

Tao Zhang

University of Science and Technology of China
zhangtaolqy@mail.ustc.edu.cn

Han Yan

Fudan University
24210240065@m.fudan.edu.cn

Jiajun Li

Huawei Technologies Ltd.
jiajun.work@huawei.com

Mingcong Song

Huawei Technologies Ltd.
songmingcong@huawei.com

Guoping Long

Huawei Technologies Ltd.
robin3@huawei.com

Hongjie Si

Huawei Technologies Ltd.
sihongjie@huawei.com

Weiwei Sun

Fudan University
wusun@fudan.edu.cn

September 29, 2026

ABSTRACT

High-performance kernels underpin efficient accelerator execution but require expert tuning and lengthy manual optimization cycles. LLM coding agents promise automation, yet their CUDA knowledge transfers poorly to data-scarce domain-specific architectures (DSAs) such as NPUs, whose execution models and memory hierarchies differ substantially from those of GPUs. To address this transfer gap, post-training methods adapt LLMs to NPU programming but depend on scarce expert data and substantial training compute. Memory-learning agents instead adapt through external memory, but their uniform credit assignment gives adopted and unused experiences the same reward target, potentially biasing subsequent retrieval rankings. Moreover, when learned values guide only retrieval, high-value experiences that generalize across operators must be retrieved repeatedly rather than retained in context, thereby increasing retrieval overhead and weakening cross-task guidance. We therefore present SAGE, a persistent self-improving agent for NPU kernel synthesis. Adoption-Traced Utility estimation (ATU) combines explicit adoption records with kernel evaluation outcomes for adoption-aware credit assignment. Utility-Gated Consolidation (UGC) uses positive utility and repeated adoption across operators to select and abstract reusable rules into a bounded resident context. On NPUScoreBench, SAGE achieves a 95.5% execution rate versus 84.1% for the strongest controlled baseline, with 86.9% of solved operators outperforming torch_npu. With GLM-5.3, SAGE achieves a $43.99\times$ speedup over the torch_npu reference on sparse flash attention. These results show that adoption-aware credit assignment and selective consolidation enable agents to accumulate and reuse hardware-specific knowledge across tasks.

1 Introduction

Large Language Models (LLMs) have made rapid progress in CUDA and Triton kernel synthesis [1, 2, 3], benefiting from mature ecosystems with abundant public kernels, documentation, and optimization idioms [4]. Such resources serve as both large-scale pre-training signals and readily available supervision or retrieval sources for post-training and augmentation, allowing LLMs to achieve strong zero-shot performance and be further specialized with relatively low cost [5]. In contrast, emerging domain-specific architectures (DSAs) such as Ascend NPUs [6, 7] provide far less public code and require architecture-specific reasoning over memory hierarchies and execution models [8]. This distribution shift is substantial: GPT-5.2 drops from 92% CUDA correctness to 14% Ascend C correctness on KernelBench L1

tasks [9, 10]. Thus, data-scarce kernel synthesis depends not only on generation but also on acquiring and reusing hardware-specific expertise under scarce supervision.

To address scarce NPU programming knowledge, post-training methods adapt LLMs through supervised fine-tuning (SFT) [11, 12] on hardware-aware data and reinforcement learning (RL) [13] with execution-based preferences [14]. Their scalability, however, is limited by scarce expert data and substantial training compute. Knowledge-augmented agents instead keep the backbone frozen and guide kernel generation and refinement with mined optimization experience [15] or structured domain knowledge [4]. Applying this knowledge to diagnose failures and revise kernels also generates new experience, raising the question of which experiences should guide subsequent tasks. Memory-learning methods in the broader agent literature address this through outcome-based values or dependency-aware credit propagation [16, 17, 18]. EvoKernel applies memory learning to NPU kernel synthesis but uses **uniform credit assignment**, a coarse-grained rule that gives every retrieved experience the same verifier reward target within an iteration [10]. A successful code repair can thus reward all top- k retrieved experiences even when the agent adopts only a small subset. Such uniform reward targets can give unused experiences favorable value estimates and higher priority in later retrieval, potentially crowding out more promising experiences with lower scores. Moreover, when learned values guide only retrieval, high-value experiences that generalize across operators must be retrieved again rather than retained in context, potentially adding retrieval overhead and limiting consistent guidance across tasks. Figure 1 contrasts *retrieval-oriented memory learning* with SAGE’s finer-grained **adoption-aware credit assignment** and selective consolidation into a bounded resident context.

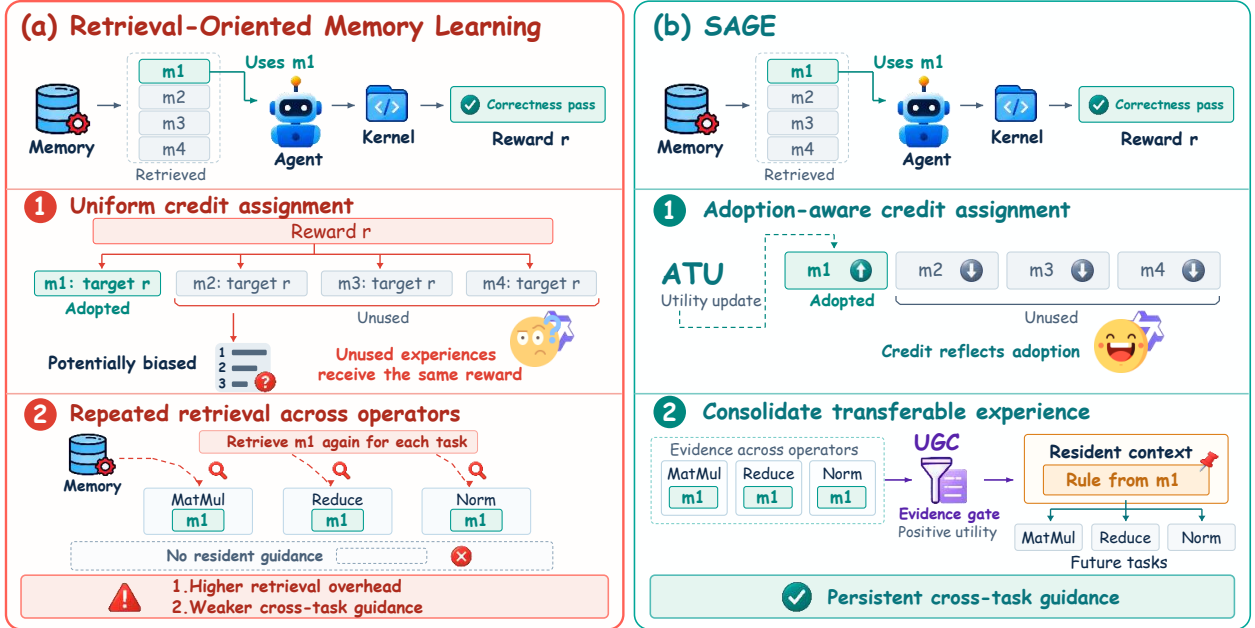


Figure 1: Retrieval-oriented memory learning with uniform credit and repeated retrieval (left) versus SAGE’s adoption-aware credit assignment and selective consolidation into resident context (right).

Our central insight is to ground credit assignment and knowledge consolidation in evidence of use. Retrieval makes an experience available; recorded adoption indicates that it guided diagnosis or code revision. Adoption-aware credit assignment combines this evidence with the kernel outcome so that unused experiences do not receive the same credit as those adopted merely for being retrieved. A single adoption, however, does not establish transferability. Positive utility accumulated through repeated adoption across distinct operators provides evidence that an experience may encode a reusable NPU programming rule. Consolidating such rules into a bounded resident context makes them available to later tasks without requiring another successful retrieval.

Building on this insight, we propose **SAGE**, a self-improvement agent for NPU kernel generation and evolution. SAGE enables a frozen backbone LLM to evolve external memory through two complementary mechanisms. Adoption-Traced Utility estimation (ATU) combines explicit adoption records with episode-level verifier outcomes to update experience utility through adoption-aware credit assignment. Utility-Gated Consolidation (UGC) leverages positive utility and repeated adoption across operators to identify experiences for consolidation into reusable resident rules, while demoting underused rules under a fixed context budget.

Our contributions are as follows:

- We introduce **SAGE**, a self-improving agent for NPU kernel synthesis that enables frozen LLMs to accumulate and reorganize hardware-specific knowledge across tasks.
- We propose **Adoption-Traced Utility estimation (ATU)**, an experience-level credit assignment mechanism that derives empirical utility estimates from adoption records.
- We propose **Utility-Gated Consolidation (UGC)**, a utility-driven memory evolution mechanism that transforms reusable experiences into persistent knowledge.
- We evaluate SAGE across three backbones on NPUIKernelBench, demonstrating consistent gains in kernel correctness and runtime performance.

2 Related Work

2.1 LLM-based Automated Kernel Synthesis

LLM-based kernel synthesis has advanced rapidly in mature GPU ecosystems. CUDA-L1 [2], CudaForge [19], KernelEvolve [20], CUDA-Agent [3], Dr.Kernel [21], and QiMeng-Kernel [22] show that LLMs can generate and iteratively optimize CUDA or Triton kernels using compiler feedback, multi-agent coordination, and hardware-in-the-loop evaluation. Their success, however, benefits from abundant public kernels, mature toolchains, and well-documented optimization idioms. For data-scarce accelerators, AscendKernelGen [14] and NKI-Agent [23] use SFT and RL to internalize Ascend C or NKI programming knowledge, but their scalability remains constrained by expert data and training compute. AscendCraft [24] instead guides generation through a lightweight DSL and structured transcompilation into Ascend C. EvoKernel [10], AscendOptimizer [15], AgenticCANN [4], Hawk [25] and AccelOpt [26] retain frozen backbones and rely on retrieved knowledge, execution feedback, and iterative repair. These approaches reduce reliance on zero-shot generation, but lack explicit adoption records to guide credit assignment and the selection of reusable knowledge for persistent context.

2.2 Memory-Augmented Agentic Self-Improvement

LLM-based agents increasingly adapt through external memory without modifying backbone parameters. Reflexion [27] converts task feedback into textual reflections retained across trials, while Voyager [28] accumulates successful behaviors in a reusable skill library. ExpeL [29] further distills transferable insights from collections of successful and failed trajectories. More recent work learns memory selection from downstream outcomes: Memento [16] trains a reward-conditioned case-selection policy, while MemRL [17] learns reward-conditioned value estimates over retrieved experiences to guide subsequent selection. MemQ [18] further propagates outcome-based credit through provenance DAGs of memory-construction dependencies. Together, these methods improve experience selection and credit propagation, but their value estimates and provenance records do not reveal which experiences within a retrieved set actually informed a code revision. SAGE instead traces adoption to assign credit to individual experiences, then uses accumulated cross-task evidence to consolidate repeatedly validated knowledge into persistent memory.

3 Methods

3.1 Preliminaries

We formulate NPU kernel synthesis over an ordered task stream under *non-parametric continual learning* [30], where a fixed generator repeatedly solves tasks while adapting only its persistent external memory. For a kernel synthesis task $\tau = (x, g)$, x denotes the operator specification and $g \in \{\text{corr}, \text{opt}\}$ denotes the task-specific objective, corresponding respectively to a correctness episode and an optimization episode, which together form its synthesis trajectory.

Let $\mathcal{G}_{\theta_0}$ denote an LLM-based kernel generator whose parameters θ_0 remain fixed throughout the task stream, and let $\mathcal{E}$ denote the compiler-and-hardware environment. At refinement round t , a generic context-construction procedure Γ derives a memory-conditioned context:

$$c_t = \Gamma(\mathcal{M}, \tau, k_{t-1}, f_{t-1}). \quad (1)$$

Here, $\mathcal{M}$ is the persistent external memory state available before processing task τ , while k_{t-1} and f_{t-1} denote the kernel implementation entering round t and its latest evaluation feedback, respectively. At $t = 1$, (k_0, f_0) denotes an initial kernel–feedback pair and defaults to $(\emptyset, \emptyset)$ when unavailable. Conditioned on these inputs, the generator samples a candidate kernel and receives execution feedback evaluated by $\mathcal{E}$:

$$k_t = \mathcal{G}_{\theta_0}(\tau, k_{t-1}, f_{t-1}, c_t), \quad f_t = \mathcal{E}(\tau, k_t). \quad (2)$$

The episode $\varepsilon = ((c_t, k_t, f_t))_{t=1}^T$ terminates when the objective specified by g is met or the interaction budget is exhausted, where T is the realized number of refinement rounds. After the episode, the external memory state is updated as

$$\mathcal{M} \leftarrow \Phi(\mathcal{M}, \varepsilon), \quad (3)$$

where Φ denotes a generic non-parametric memory-update procedure. Repeated application of this transition across successive tasks constitutes the continual learning process. The objective is to improve subsequent execution outcomes solely via these memory transitions while θ_0 remains fixed.

3.2 SAGE Framework

As shown in Figure 2, SAGE draws inspiration from hierarchical memory management for fixed-context LLM agents [31] and instantiates the generic memory state $\mathcal{M}$ as a cold-hot hierarchy with associated evidence statistics:

$$\mathcal{M} = (\mathcal{C}, \mathcal{H}), \quad \text{Tok}(\mathcal{H}) \leq B, \quad (4)$$

where $\mathcal{C}$ denotes *cold memory*, which preserves source experiences and their item-level evidence and lifecycle statistics, while $\mathcal{H}$ denotes *hot memory*, which provides consolidated rules to the generator’s context without retrieval. Here, B is the fixed token budget. At the end of each task, SAGE writes newly acquired experiences to cold memory $\mathcal{C}$ for retrieval in subsequent tasks.

SAGE realizes the memory transition Φ through two complementary mechanisms:

- **Adoption-Traced Utility Estimation (ATU).** ATU uses adoption records to distribute terminal verifier credit among adopted experiences and penalize unused retrievals, thereby updating empirical utility estimates.
- **Utility-Gated Consolidation (UGC).** UGC uses positive utility and repeated adoption across operators to identify experiences for rule abstraction. It selects resident rules under a fixed token budget, allowing persistently underused rules to be displaced.

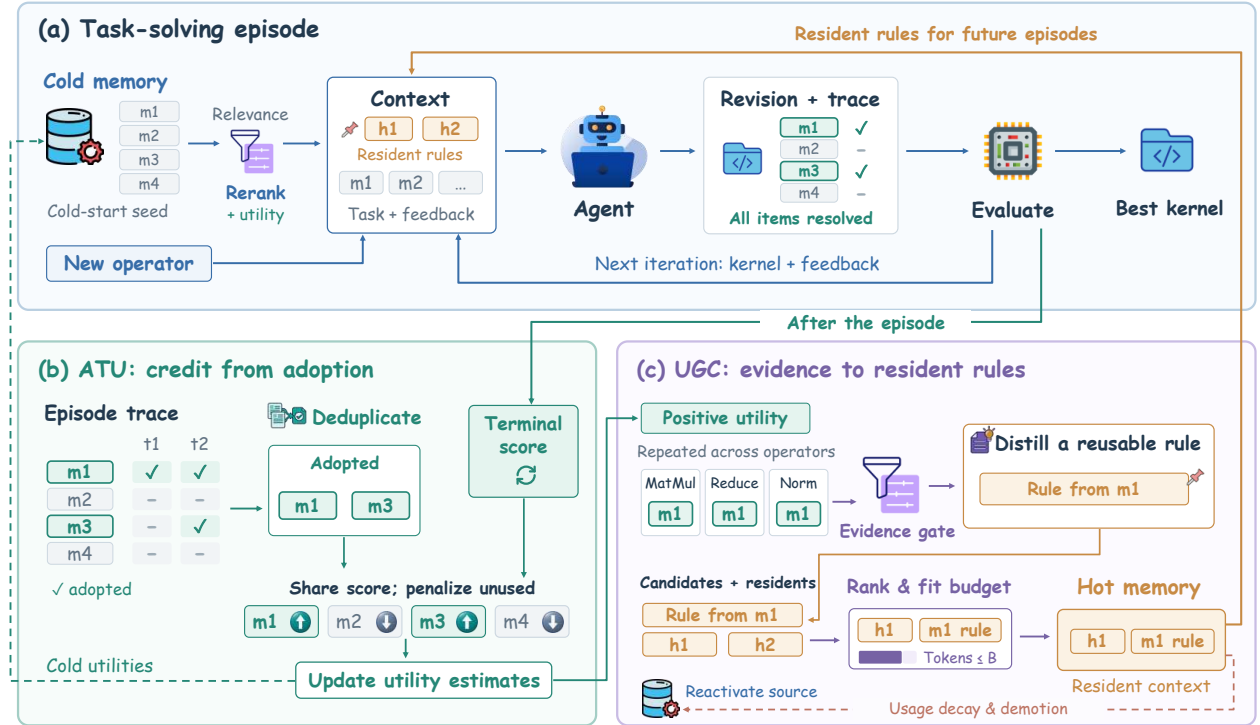


Figure 2: **Overview of SAGE.** (a) A frozen agent iteratively refines kernels using retrieved cold experiences, resident hot rules, and execution feedback. (b) ATU combines adoption traces with terminal outcomes to estimate item-level utility. (c) UGC uses utility and repeated adoption across operators to select reusable rules for a bounded resident context and demote underused rules.

3.3 Adoption-Traced Utility Estimation

Compiler and device feedback is available after every generation, but it evaluates the resulting kernel rather than the textual experiences that shaped it. Uniform credit assignment applies the same reward target to all retrieved experiences, although only a subset may guide diagnosis or code revision. ATU performs adoption-aware credit assignment by combining explicit adoption records with terminal verifier outcomes to distinguish adopted experiences from unused retrievals.

Trace-Grounded Experience Collection. For task τ , let $\mathcal{R}_t \subseteq \mathcal{C}$ denote the experiences retrieved at refinement round t . For each item $m \in \mathcal{R}_t$, the agent records an adoption state $a_{m,t} \in \{1, 0, \perp\}$ (adopted, not adopted, or unresolved) and a rationale. To ensure complete adoption records, SAGE inserts a *trace-completeness hook* in the evaluation script that blocks evaluation until the adoption state of every m is explicitly resolved. Appendix D.1 reports an expert audit assessing whether recorded adoption corresponds to actual code revisions. For each item, SAGE maintains deduplicated retrieval and adoption counts, n_m^{ret} and n_m^{ado} , and the set $\mathcal{O}_m$ of distinct adopting operators.

Adoption-Aware Credit Assignment. The benefit of an adopted experience may emerge only in the final kernel; ATU therefore defers credit assignment until episode termination. For an optimization episode, let T_{first} and T_{best} denote the execution latencies of the first correct kernel and the best correct kernel found within the episode, respectively. We define the terminal score as

$$z = \begin{cases} 1, & g = \text{corr and correctness passes,} \\ 1 - \frac{T_{\text{best}}}{T_{\text{first}}}, & g = \text{opt and } T_{\text{best}} < T_{\text{first}}, \\ -\delta, & \text{otherwise,} \end{cases} \quad (5)$$

where $0 < \delta < 1$ is a small failure penalty. The optimization score measures the relative latency reduction achieved after correctness is first established and is naturally bounded in $(0, 1)$. Let $\mathcal{R} = \bigcup_{t=1}^T \mathcal{R}_t$ and $\mathcal{A} = \{m \in \mathcal{R} : \exists t, a_{m,t} = 1\}$ denote the retrieved and adopted experiences over episode ε . ATU converts the terminal score into item-level evidence:

$$\xi_m = \begin{cases} \frac{z}{|\mathcal{A}|}, & m \in \mathcal{A}, \\ -\beta, & m \in \mathcal{R} \setminus \mathcal{A}, \end{cases} \quad (6)$$

where $0 < \beta < 1$ controls the penalty for retrieved but unused experiences. Hence, adopted items share a fixed episode-level credit budget, preventing credit inflation when multiple experiences are adopted, whereas non-adopted retrievals receive a fixed negative credit.

Online Utility Tracking and Retrieval. Each memory item maintains an empirical utility $u_m \in [-1, 1]$, initialized at zero and updated online via the following *bandit-style* rule:

$$u_m \leftarrow (1 - \eta_m)u_m + \eta_m \xi_m, \quad \eta_m = \max\left(\frac{1}{1 + n_m^{\text{ret}}}, \eta_{\min}\right). \quad (7)$$

The decaying step size stabilizes utility estimates as evidence accumulates, while $0 < \eta_{\min} \ll 1$ prevents historical evidence from permanently dominating subsequent utility updates. Thus, u_m estimates retrieval-conditioned empirical utility and subsequently guides experience ranking and memory consolidation.

At refinement round t , SAGE first uses a fixed hybrid sparse–dense retriever to obtain an over-retrieved candidate pool $\tilde{\mathcal{R}}_t$ from cold memory based solely on relevance. Utility is incorporated only during late reranking:

$$s(m) = s_{\text{rel}}(m)(1 + \gamma u_m), \quad m \in \tilde{\mathcal{R}}_t, \quad (8)$$

where $s_{\text{rel}}(m)$ is the relevance score and $0 \leq \gamma < 1$ controls the strength of utility modulation. The highest-ranked experiences form the final retrieval set $\mathcal{R}_t$ used for context construction and subsequent adoption tracing, while resident knowledge in $\mathcal{H}$ bypasses retrieval.

3.4 Utility-Gated Consolidation

High-value experiences that generalize across operators should remain available in future task contexts without repeated retrieval. Utility-Gated Consolidation (UGC) uses positive utility and repeated adoption across operators to identify candidates for rule abstraction, then selects resident rules under a fixed context budget and allows underused rules to be displaced. UGC is invoked after each task-solving episode, following the ATU utility update.

Evidence-Gated Admission and Abstraction. A cold-memory item becomes eligible for consolidation only after positive utility, sufficient adoption and cross-operator validation:

$$u_m > 0, \quad n_m^{\text{ado}} \geq K_{\text{ado}}, \quad |\mathcal{O}_m| \geq K_{\text{op}}, \quad K_{\text{ado}}, K_{\text{op}} > 1. \quad (9)$$

Each source experience maintains a consolidation state $\sigma_m \in \{\text{normal, validated, consolidated}\}$. An item satisfying the above gates transitions from normal to validated, and the agent abstracts it into a concise transferable rule, optionally with a minimal code or API example, while preserving its source identifier. The source experience and its accumulated evidence are retained for provenance, but the source is excluded from active retrieval while its consolidated rule remains resident. Source and rule share one identifier and utility u_m ; the rule has no separate value estimator.

Budgeted Hot-Memory Selection. Let $\mathcal{V} \subseteq \mathcal{C}$ denote the validated cold-memory candidates eligible for consolidation. UGC ranks items in $\mathcal{H} \cup \mathcal{V}$ by expected utility per resident token:

$$U(m) = \frac{u_m \hat{p}_m}{L_m}, \quad \hat{p}_m = \frac{n_m^{\text{ret}}}{N_m^{\text{elig}}}, \quad m \in \mathcal{H} \cup \mathcal{V}, \quad (10)$$

where $\hat{p}_m$ is an empirical access-frequency estimate, L_m is the resident token cost, and N_m^{elig} counts the episodes in which m is active in cold memory and available to the retriever. Once item m is written to cold memory, N_m^{elig} increases once per such episode, while n_m^{ret} increases only when m is retrieved. The hot-memory set is greedily selected in descending $U(m)$ subject to $\text{Tok}(\mathcal{H}) \leq B$, where B is the fixed token budget. The complete replacement procedure is given in Algorithm 1.

Resident items bypass retrieval, so the *trace-completeness hook* introduced in Sec. 3.3 additionally records their episode-level usage. Their access-frequency estimates are updated by usage as

$$\hat{p}_m \leftarrow (1 - \mu)\hat{p}_m + \mu \mathbf{1}_{m \text{ is used}}, \quad 0 < \mu < 1. \quad (11)$$

Consequently, persistent non-use decreases $\hat{p}_m$ and hence $U(m)$. Such items may be demoted, upon which their source experiences regain retrieval eligibility. UGC thus enables bidirectional cold-hot evolution under a fixed token budget rather than monotonic context growth.

4 Experiments

4.1 Experimental Setup

Benchmark and Execution. We evaluate SAGE on NPUKernelBench [14], which covers NPU kernel synthesis across multiple operator categories and complexity levels. We extend the benchmark with iterative agent evaluation, per-round execution traces, and support for custom operators beyond the built-in tasks. All kernels are compiled and evaluated on Ascend 910C NPUs under a fixed CANN software stack.

Models and Budget. We use OpenCode [32] as the agent harness and evaluate three source models spanning different scales: Qwen3-Coder-Next [33], DeepSeek-V4-Flash [34] and GLM-5.2 [35]. All methods share the same prompts, cold-start knowledge, CANN diagnostic tools, retrieval pipeline, and hardware evaluator. We allow at most $T = 30$ refinement rounds per operator, shared across its correctness and optimization episodes.

Metrics. Following NPUKernelBench, we report **Compilation Rate (CR)** and **Execution Rate (ER)**, the fractions of operators for which a compilable and functionally correct kernel is found within the interaction budget, respectively. Let $\mathcal{O}$ denote the solved evaluation operators and $T_{\text{torch}}(o)$ the latency of the corresponding `torch_npu` implementation; $T_{\text{first}}(o)$ and $T_{\text{best}}(o)$ follow the episode-level definitions in Sec. 3.3. We report

$$\text{Fast}_{1.0} = \frac{1}{|\mathcal{O}|} \sum_{o \in \mathcal{O}} \mathbf{1}_{T_{\text{torch}}(o) > T_{\text{best}}(o)}, \quad S_{\text{self}} = \text{median}_{o \in \mathcal{O}} \frac{T_{\text{first}}(o)}{T_{\text{best}}(o)}. \quad (12)$$

$\text{Fast}_{1.0}$ measures the fraction of solved operators outperforming their `torch_npu` counterparts, while S_{self} measures performance gain achieved through iterative refinement.

Baselines. Benchmark differences and limited access to complete implementations and knowledge bases hinder direct system-level comparisons with existing state-of-the-art methods. We therefore compare SAGE with three controlled baselines under the same OpenCode agent harness:

- **Refinement** performs iterative repair and optimization within each operator but retains no cross-task experience;

- **Static RAG** uses the same initial cold-memory repository and retrieval pipeline as SAGE but disables subsequent memory writes and utility updates;
- **Value Memory** adapts EvoKernel’s value-driven cross-task memory [10] from its OpenReview supplementary material, using uniform credit assignment to give co-retrieved experiences the same verifier reward target for subsequent retrieval.

4.2 Main Results

NPUKernelBench exhibits a highly imbalanced distribution across operator categories. We therefore construct a coverage-oriented subset of 88 operators: 20 Level-1 and 50 Level-2 tasks covering all categories at their respective levels, together with all 18 Level-3 tasks. Evaluation follows a sequential learning stream, with each task’s result recorded before any memory updates for subsequent tasks. The complete operator list is provided in Appendix A.1. Table 1 summarizes the results across three source models and four memory settings.

Table 1: Main results on the 88-operator NPUKernelBench subset. All rates are reported as percentage. The best result for each model is marked in bold.

Model	Method	Execution Rate			Overall		Performance	
		L1	L2	L3	CR	ER	Fast _{1.0}	S _{self}
Qwen3-Coder-Next (80B)	Refinement	5.0	0.0	0.0	35.2	1.1	0.0	1.00×
	Static RAG	30.0	26.0	0.0	51.1	21.6	21.1	1.10×
	Value Memory	50.0	24.0	5.6	65.9	26.1	17.4	1.16×
	SAGE	55.0	32.0	5.6	75.0	31.8	28.6	1.27×
DeepSeek-V4-Flash (284B)	Refinement	35.0	18.0	0.0	55.7	18.2	31.3	1.20×
	Static RAG	85.0	30.0	5.6	73.9	37.5	42.4	1.42×
	Value Memory	80.0	58.0	27.8	89.8	56.8	50.0	1.67×
	SAGE	90.0	62.0	44.4	95.5	64.8	64.9	2.08×
GLM-5.2 (744B)	Refinement	65.0	46.0	5.6	79.5	42.0	48.6	1.74×
	Static RAG	75.0	60.0	38.9	88.6	59.1	59.6	2.12×
	Value Memory	95.0	88.0	61.1	97.7	84.1	75.7	3.36×
	SAGE	100.0	96.0	88.9	98.9	95.5	86.9	4.47×

Overall performance. SAGE achieves the strongest results across all three backbones and evaluation dimensions. With GLM-5.2, it solves 84 of 88 operators within the interaction budget, while most solved kernels outperform their corresponding `torch_npu` implementations. Relative to Value Memory, Fast_{1.0} increases from 75.7% to 86.9%, while median S_{self} improves from 3.36× to 4.47×. The gains persist on DeepSeek-V4-Flash and Qwen3-Coder-Next, indicating that SAGE is not tied to a particular backbone.

Effect of memory evolution. The progression from Refinement to Static RAG, Value Memory, and SAGE shows consistent overall gains from increasingly structured cross-task experience reuse, despite local non-monotonicity. The advantage becomes more pronounced on harder tasks: SAGE raises Level-3 ER over Value Memory from 61.1% to 88.9% with GLM-5.2 and from 27.8% to 44.4% with DeepSeek-V4-Flash. This widening gap suggests that reliable experience evaluation and consolidation become increasingly important as task complexity grows.

Backbone and optimization behavior. Backbone capability strongly affects how effectively accumulated knowledge is exploited. Under SAGE, overall ER decreases from 95.5% with GLM-5.2 to 64.8% with DeepSeek-V4-Flash and 31.8% with Qwen3-Coder-Next, with an even larger separation on Level-3 tasks. GLM-5.2 nevertheless exhibits a long-tailed optimization distribution, with speedups ranging from **0.38×** to **71.41×**, **motivating the use of Fast_{1.0} together with median S_{self}** rather than arithmetic-mean speedup. Per-operator speedups relative to `torch_npu` for the correctness-passing Level-3/GMM tasks with GLM-5.2 are reported in Appendix A.3.

4.3 Transfer Beyond the Benchmark

To examine whether the evolved memory remains effective beyond NPUKernelBench, we further evaluate GLM-5.2 on 16 substantially more complex xCCL operators involving fused attention, normalization, cache management, and

GMM-based computation. The complete operator list is provided in Appendix A.2. Memory-based methods start from their post-benchmark memory states while retaining their native memory-update behavior, and all methods process the xCCL operators in the same fixed order under the same interaction budget. Thus, memory is allowed to continue evolving as the system adapts to the new operator library. This evaluates adaptation to a new operator library from evolved memory, combining source-memory reuse with continued learning on xCCL.

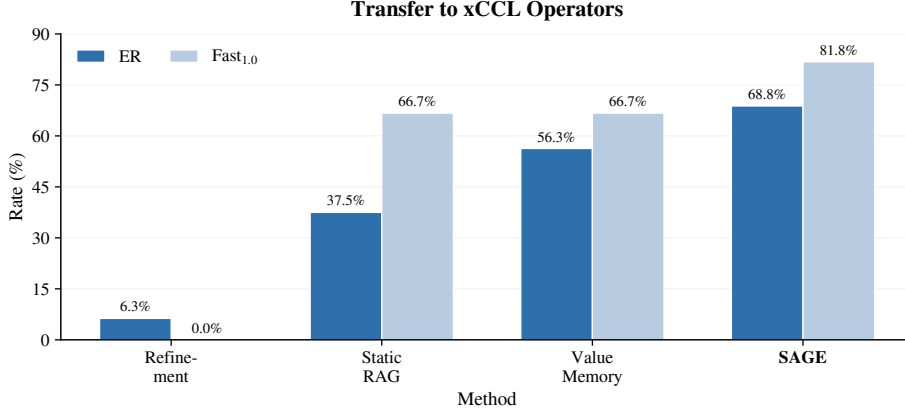


Figure 3: Transfer results on xCCL with GLM-5.2; memory updates remain enabled.

As shown in Figure 3, performance drops markedly for all methods relative to NPUKernelBench, reflecting the greater complexity and distribution shift of these operators. SAGE nevertheless solves 11 of 16 operators, achieving 68.8% ER. Among these correct kernels, 81.8% outperform their corresponding `torch_npu` implementations.

We further evaluate SAGE with GLM-5.3 on the same 16 xCCL operators, observing an ER increase from 68.8% to 81.3% and substantial performance gains across all operators solved by GLM-5.2. For sparse flash attention, a key kernel for efficient long-context LLM inference, speedup over `torch_npu` increases from $23.35\times$ to $43.99\times$. Its complete kernel code is provided as an example in Appendix F.3. These results highlight SAGE’s optimization potential on deployment-relevant kernels and provide further evidence that backbone capability substantially affects generated kernel performance, complementing the findings in Section 4.2. Per-operator results for both backbones are provided in Appendix A.4.

4.4 Effect of Task Order

To examine how task order affects kernel synthesis, we use a fixed subset of 25 Level-2 operators as source tasks and all 18 Level-3 operators as targets. Then we compare three task schedules:

- **L3 Only:** SAGE processes only the Level-3 operator stream.
- **L2+L3 Mixed:** SAGE processes a fixed interleaving of the Level-2 and Level-3 operators.
- **L2→L3:** SAGE first processes all Level-2 operators and then continues on the Level-3 stream using the resulting memory.

All schedules start from identical memory, use the same per-operator interaction budget, update memory after each task-solving episode, and never revisit operators. We report only Level-3 results, recorded before incorporating each operator’s experience.

Table 2: Cross-difficulty transfer to the 18 Level-3 operators. All rates are percentages.

Task schedule	L3 ER	Fast _{1.0}
L3 Only	38.9	42.9
L2+L3 Mixed	61.1	63.6
L2→L3	77.8	71.4

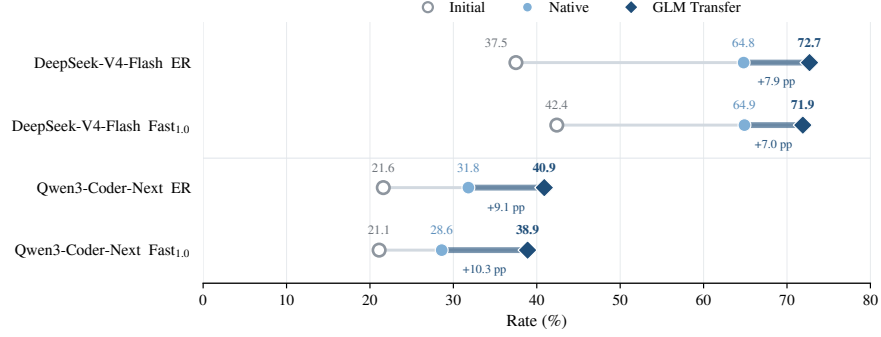


Figure 4: Cross-backbone transfer of initial, native-evolved, and GLM-5.2-evolved SAGE memory on the same read-only evaluation set.

The sequential $L2 \rightarrow L3$ schedule achieves the strongest performance, improving L3 ER by 16.7 percentage points over the mixed stream and by 38.9 points over $L3$ -only adaptation. A consistent ordering is observed for Fast_{1.0}. This progression indicates that consolidating lower-difficulty experience in advance provides a stronger basis for both functional synthesis and performance optimization on harder operators.

4.5 Cross-Backbone Memory Transfer

The backbone dependence observed in Section 4.2 raises a natural question: does SAGE accumulate model-specific experience, or can the resulting knowledge be exploited by a different generator? We use GLM-5.2 as the memory donor and evaluate DeepSeek-V4-Flash and Qwen3-Coder-Next under three conditions: the shared initial memory before continual adaptation (*Initial*), memory evolved by the recipient itself (*Native*), and the complete SAGE memory evolved by GLM-5.2 on the same learning stream (*GLM Transfer*). Each condition independently reruns generation and hardware evaluation on the same 88 read-only operators with the same cold-start knowledge, diagnose tools and interaction budget; no memory updates are performed during transfer evaluation.

As shown in Figure 4, both recipient backbones benefit from memory evolved by GLM-5.2. For DeepSeek-V4-Flash, GLM Transfer raises ER from 64.8% with native memory to 72.7%, while Fast_{1.0} increases from 64.9% to 71.9%. Qwen3-Coder-Next also improves, with ER increasing from 31.8% to 40.9% and Fast_{1.0} from 28.6% to 38.9%. These gains, obtained without parameter updates or additional memory evolution, indicate that the knowledge accumulated by SAGE is partially portable across backbone models. Nevertheless, the substantial performance gap to GLM-5.2 remains, suggesting that transferred memory provides complementary domain knowledge rather than compensating for differences in backbone capability.

4.6 Ablation Study

We ablate SAGE with GLM-5.2 on the same 88 operators. *Uniform Credit* removes adoption tracing and assigns every retrieved experience the same terminal episode score, regardless of its adoption. *w/o UGC* retains ATU and utility-aware retrieval but disables cold-hot consolidation, while *Random UGC* randomly selects eligible resident rules under the same token budget as SAGE.

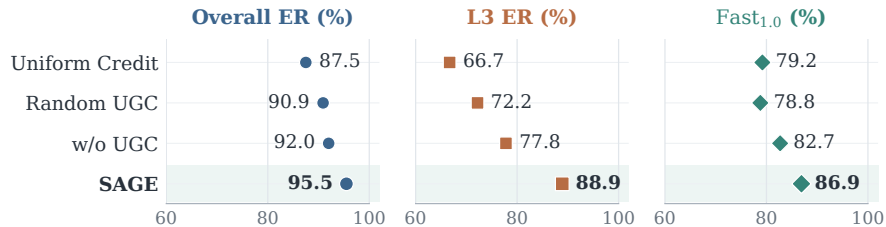


Figure 5: Ablation with GLM-5.2 on 88 operators. All metrics are percentages; Fast_{1.0} is conditional on correctness-passing operators.

Figure 5 shows complementary benefits from adoption-aware credit assignment and utility-guided consolidation. Uniform Credit yields the largest L3 ER drop (88.9% to 66.7%). This suggests that coarse-grained attribution is

especially harmful on complex kernel synthesis trajectories. Removing UGC retains much of the benefit of ATU but remains below full SAGE, while Random UGC performs worse than utility-guided consolidation. Together, these results indicate that SAGE benefits not merely from accumulating more context, but from identifying useful experience and selectively converting it into persistent knowledge.

5 Conclusion

We presented SAGE, a persistent self-improving agent for NPU kernel synthesis that enables frozen LLMs to accumulate hardware-specific expertise through external memory evolution. Rather than treating retrieved experiences uniformly, SAGE traces experience adoption to estimate item-level utility and consolidates repeatedly validated knowledge under a bounded hot-memory budget. Across three backbones, SAGE improves both correctness and efficiency, with larger gains on complex Level-3 operators; transfer results and mechanism ablations support the role of accurate attribution and selective consolidation. However, our 30-round study does not cover long-horizon synthesis of fused kernels and megakernels, which requires diversity-preserving search across coupled scheduling, memory-layout, and synchronization decisions. In practice, human kernel programmers may use isolated SAGE sandboxes or jointly update shared cold and hot memories, raising issues of specialization, provenance, transfer, and conflict resolution. Future work will explore this principle across other emerging hardware architectures and domain-specific programming languages.

References

- [1] Yang Yu, Peiyu Zang, Chi Hsu Tsai, Haiming Wu, Yixin Shen, Jialing Zhang, Haoyu Wang, Zhiyou Xiao, Jingze Shi, Yuyu Luo, et al. Towards automated kernel generation in the era of llms. *arXiv preprint arXiv:2601.15727*, 2026.
- [2] Xiaofei Sun, Albert Wang, Jiwei Li, Chris Shum, et al. Cuda-ll: Improving cuda optimization via contrastive reinforcement learning. In *International Conference on Learning Representations*, volume 2026, pages 123922–123957, 2026.
- [3] Weinan Dai, Hanlin Wu, Qiying Yu, Huan-ang Gao, Jiahao Li, Chengquan Jiang, Weiqiang Lou, Yufan Song, Hongli Yu, Jiaze Chen, et al. Cuda agent: Large-scale agentic rl for high-performance cuda kernel generation. *arXiv preprint arXiv:2602.24286*, 2026.
- [4] Junhao Qiu, Zidong Wang, Yansong Sun, Zhitong Ma, Ping Guo, and Qingfu Zhang. Agenticcann: Automated ascend c operator generation via knowledge-augmented agentic evolution. *arXiv preprint arXiv:2607.26661*, 2026.
- [5] Shushan Arakelyan, Rocktim Das, Yi Mao, and Xiang Ren. Exploring distributional shifts in large language models for code analysis. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 16298–16314, 2023.
- [6] Cristina Silvano, Daniele Ielmini, Fabrizio Ferrandi, Leandro Fiorin, Serena Curzel, Luca Benini, Francesco Conti, Angelo Garofalo, Cristian Zambelli, Enrico Calore, et al. A survey on deep learning hardware accelerators for heterogeneous hpc platforms. *ACM Computing Surveys*, 57(11):1–39, 2025.
- [7] Heng Liao, Jiajin Tu, Jing Xia, Hu Liu, Xiping Zhou, Honghui Yuan, and Yuxing Hu. Ascend: a scalable and unified architecture for ubiquitous deep neural network computing: Industry track paper. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 789–801. IEEE, 2021.
- [8] Yuhang Zhou, Zhibin Wang, Guyue Liu, Shipeng Li, Xi Lin, Zibo Wang, Yongzhong Wang, Fuchun Wei, Jingyi Zhang, Zhiheng Hu, et al. Squeezing operator performance potential for the ascend architecture. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 1156–1171, 2025.
- [9] Anne Ouyang, Simon Guo, Simran Arora, Alex L Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. Kernelbench: Can llms write efficient gpu kernels? *arXiv preprint arXiv:2502.10517*, 2025.
- [10] Yujie Zheng, Zhuo Li, Shengtao Zhang, Hanjing Wang, Junjie Sheng, Jiaqian Wang, Junchi Yan, Weinan Zhang, Ying Wen, Bo Tang, et al. Towards cold-start drafting and continual refining: A value-driven memory approach with application to npu kernel synthesis. *arXiv preprint arXiv:2603.10846*, 2026.
- [11] Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. Lima: Less is more for alignment. *Advances in Neural Information Processing Systems*, 36:55006–55021, 2023.

- [12] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. Scaling instruction-finetuned language models. *Journal of machine learning research*, 25(70):1–53, 2024.
- [13] Zihan Zhang, Yuxin Chen, Jason Lee, and Simon S Du. Settling the sample complexity of online reinforcement learning. *Journal of the ACM*, 72(3):1–63, 2025.
- [14] Xinzi Cao, Jianyang Zhai, Pengfei Li, Zhiheng Hu, Cen Yan, Bingxu Mu, Guanghuan Fang, Bin She, Jiayu Li, Yihan Su, et al. Ascendkernelgen: A systematic study of llm-based kernel generation for neural processing units. *arXiv preprint arXiv:2601.07160*, 2026.
- [15] Jiehao Wu, Zixiao Huang, Wenhao Li, Chuyun Shen, Junjie Sheng, and Xiangfeng Wang. Ascendoptimizer: Episodic agent for ascend npu operator optimization. *arXiv preprint arXiv:2603.23566*, 2026.
- [16] Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, et al. Memento: Fine-tuning llm agents without fine-tuning llms. *arXiv preprint arXiv:2508.16153*, 2025.
- [17] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, et al. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. *arXiv preprint arXiv:2601.03192*, 2026.
- [18] Junwei Liao, Haoting Shi, Ruiwen Zhou, Jiaqian Wang, Shengtao Zhang, Wei Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Bo Tang, et al. Memq: Integrating q-learning into self-evolving memory agents over provenance dags. *arXiv preprint arXiv:2605.08374*, 2026.
- [19] Zijian Zhang, Rong Wang, Shiyang Li, Yuebo Luo, Mingyi Hong, and Caiwen Ding. Cudaforge: An agent framework with hardware feedback for cuda kernel optimization. *arXiv preprint arXiv:2511.01884*, 2025.
- [20] Gang Liao, Hongsen Qin, Ying Wang, Alicia Golden, Michael Kuchnik, Yavuz Yetim, Ruichao Xiao, Jia Jiunn Ang, Chunli Fu, Yihan He, et al. Kernelevolve: Scaling agentic kernel coding for heterogeneous ai accelerators at meta. In *2026 ACM/IEEE 53rd Annual International Symposium on Computer Architecture (ISCA)*, pages 733–747. IEEE, 2026.
- [21] Wei Liu, Jiawei Xu, Yingru Li, Longtao Zheng, Tianjian Li, Qian Liu, and Junxian He. Dr. kernel: Reinforcement learning done right for triton kernel generations. *arXiv preprint arXiv:2602.05885*, 2026.
- [22] Xinguo Zhu, Shaohui Peng, Jiaming Guo, Yunji Chen, Qi Guo, Yuanbo Wen, Hang Qin, Ruizhi Chen, Qirui Zhou, Ke Gao, et al. Qimeng-kernel: Macro-thinking micro-coding paradigm for llm-based high-performance gpu kernel generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 29168–29176, 2026.
- [23] Junjie Tang, Jun Huan, Hao Zhou, Yuhao Zhang, and Lin Wang. Nki-agent: Domain-specific fine-tuning and agentic tool use for neuron kernel generation. *arXiv preprint arXiv:2607.04395*, 2026.
- [24] Zhongzhen Wen, Shudi Shao, Zhong Li, Yu Ge, Tongtong Xu, Yuanyi Lin, and Tian Zhang. Ascendcraft: Automatic ascend npu kernel generation via dsl-guided transcompilation. *arXiv preprint arXiv:2601.22760*, 2026.
- [25] Junyi Wen, Ruiyan Zhuang, Yongjia Xu, Pengtu Li, Rui Zou, Hongyi Chen, Chingman Wan, Puxu Yang, Wuhui Chen, and Yanlin Wang. Hawk: Harnessing hardware-aware knowledge for high-performance npu kernel generation. *arXiv preprint arXiv:2607.01590*, 2026.
- [26] Genghan Zhang, Shaowei Zhu, Anjiang Wei, Zhenyu Song, Allen Nie, Zhen Jia, Nandita Vijaykumar, Yida Wang, and Kunle Olukotun. Accelopt: A self-improving llm agentic system for ai accelerator kernel optimization. *Proceedings of Machine Learning and Systems*, 8:541–568, 2026.
- [27] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- [28] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. URL <https://arxiv.org/abs/2305.16291>, 2(11), 2023.
- [29] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642, 2024.
- [30] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From rag to memory: Non-parametric continual learning for large language models. *arXiv preprint arXiv:2502.14802*, 2025.

- [31] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [32] OpenCode Contributors. OpenCode: The open source coding agent. <https://github.com/anomalyco/opencode>, 2026. Software repository.
- [33] Ruisheng Cao, Mouxiang Chen, Jiawei Chen, Zeyu Cui, Yunlong Feng, Binyuan Hui, Yuheng Jing, Kaixin Li, Mingze Li, Junyang Lin, et al. Qwen3-coder-next technical report. *arXiv preprint arXiv:2603.00729*, 2026.
- [34] Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, et al. Deepseek-v4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026.
- [35] Z.ai. Glm-5.2: Built for long-horizon tasks. <https://z.ai/blog/glm-5.2>, 2026.

A Evaluation Operator Lists

A.1 NPUKernelBench Evaluation Subset

Table 3 lists the 88 NPUKernelBench operators used in our main evaluation, comprising all Level-3 tasks and coverage-oriented subsets of Levels 1 and 2 from the full benchmark.

Table 3: Operators included in the 88-task NPUKernelBench evaluation subset.

Level	Category	Operators
Level 1	Comparison	Equal, Less, LessEqual
	Condition	IsFinite, IsInf, NonFiniteCheckOp
	Index	GatherV3, ScatterList
	Math	Sqrt, SinMath, CastMath, AbsMath, Addcdiv, ClipByValue, CosMath, Lerp, StridesliceNegConcatV2
	TensorCreation	Arange, Eye, Fill
Level 2	Activation	DequantSwigluQuant, FastGelu, Gelu, InplaceAttnSoftmax, SwiGlu, Swish, GeGluV2, GeluGrad, MulSigmoid
	Foreach	ForeachAbs, ForeachAddScalar, ForeachMulScalar, ForeachDivScalar, ForeachExp, ForeachSqrt, ForeachNeg, ForeachZeroInplace, ForeachAddList, ForeachMulList, ForeachSubScalar, ForeachCos, ForeachSigmoid
	Linalg	Cross
	Loss	CrossEntropyLoss, MseLoss, CrossEntropyLossGrad, MseLossGradV2
	Mask	MaskedSelectV3, Tril, Triu
	Norm	LayerNormV4, RmsNorm, DeepNorm, BatchNormV3, GroupNormSwish, AddLayerNorm, PreLayerNorm, AddRmsNorm, InplaceAddRmsNorm, GroupNormSwishGrad
	Optim	ApplyAdamWV2, ApplyFusedEmaAdam
	Reduce	AddSigmoidMulReduceSumD, SelectReduceMaxDSubExpReduceSumDRealDiv, MulMulReduceMeanDTwice, MulSigmoidMulAddCustom
Level 3	TensorMove	CoalesceSparse, StridedSliceAssignV2, ExpandV2, ReverseSequence
	GMM	BasicMatmul, BatchedMatmul, FlashAttentionScoreWithLargeHeadDim, Gemm, Gemv, GroupGemm, GroupedMatmulSliceK, GroupedMatmulSliceKPerTokenDequant, GroupedMatmulSliceM, GroupedMatmulSliceMPerTokenDequant, MatmulAdd, MatmulBias, Mla, OptimizedMatmul, PaddingMatmul, QuantMatmul, SplitkMatmul
	Sort	TopKV3

A.2 xCCL Transfer Evaluation Set

Table 4 lists the 16 xCCL operators used for the cross-library transfer evaluation in Section 4.3.

Table 4: The 16 xCCL operators used in the transfer evaluation.

Operator	Operator
lightning_indexer	sparse_flash_attention
mha_prolog_v3_x	compute_and_cache_digest
apply_rotary_pos_emb_partial	gmm_deq_swiglu_quant_gmm_deq
hc_post_x	hc_post
fused_gelu_sigmoid	fused_scatter_update_and_compute_digest
rms_norm_rope	grouped_matmul_swiglu_quant_x
moe_dist_dispatch_mul_exp	moe_gating_top_k_remap
rotary_pos_emb_partial	dequant_swiglu_clamp_quant

A.3 Level-3/GMM Per-Operator Speedups

Figure 6 reports the best correct SAGE kernel’s speedup over `torch_npu` for each of the 16 correctness-passing Level-3/GMM operators. The median speedup is $1.54\times$; individual speedups span $0.38\times$ to $71.41\times$, illustrating why a single aggregate cannot describe the optimization distribution. Bars show $\log_2$ speedup, while their annotations retain the original multiplicative values.

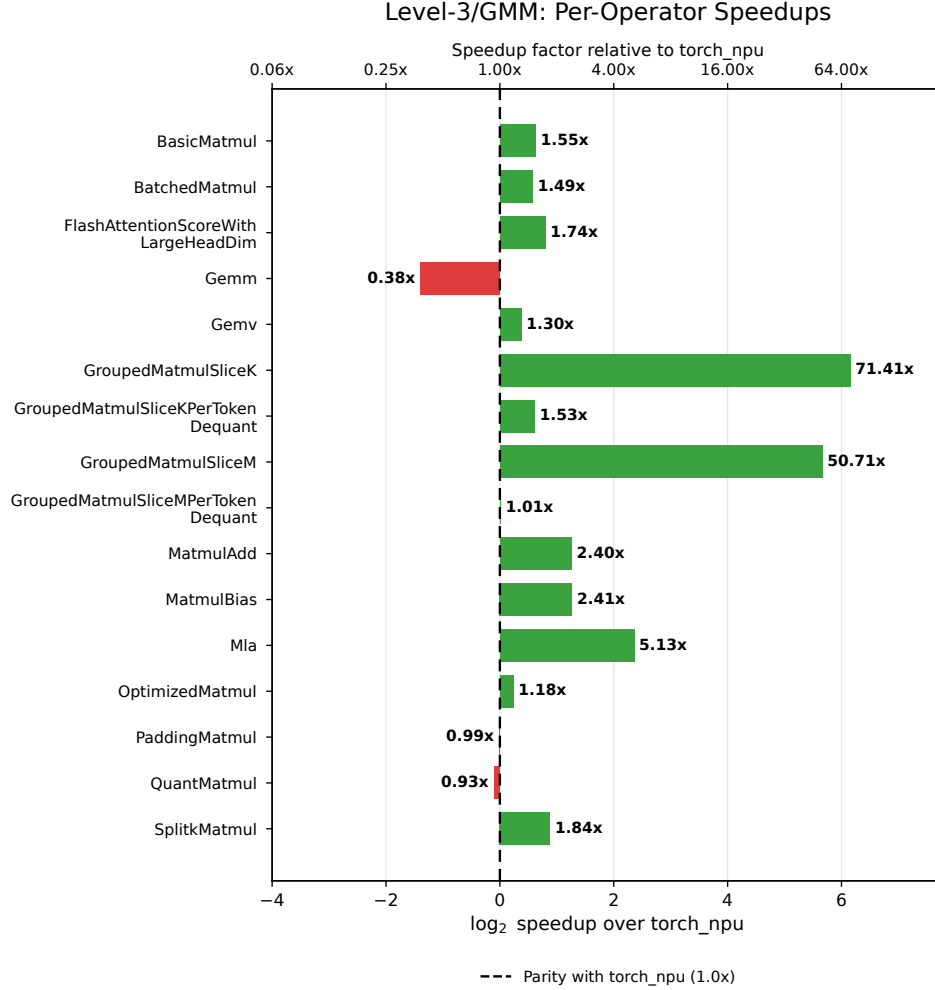


Figure 6: Per-operator speedups relative to `torch_npu` for correctness-passing Level-3/GMM tasks with GLM-5.2. Green and red indicate speedups above and below $1\times$, respectively; the dashed line marks parity. Horizontal distances are $\log_2$ -transformed.

A.4 xCCL Per-Operator Speedups

Figure 7 reports per-operator speedups for the xCCL tasks solved by either GLM-5.2 or GLM-5.3. On the 11 operators solved by both backbones, the median speedup over `torch_npu` increases from $3.32\times$ with GLM-5.2 to $7.32\times$ with GLM-5.3. GLM-5.3 additionally solves `rms_norm_rope` and `rotary_pos_emb_partial`, achieving $11.81\times$ and $10.86\times$ speedups, respectively; its median over all 13 correctness-passing xCCL operators is $10.86\times$. Both evaluations inherit memory evolved over all 88 NPUKernelBench tasks, providing a mature starting point for cross-library adaptation.

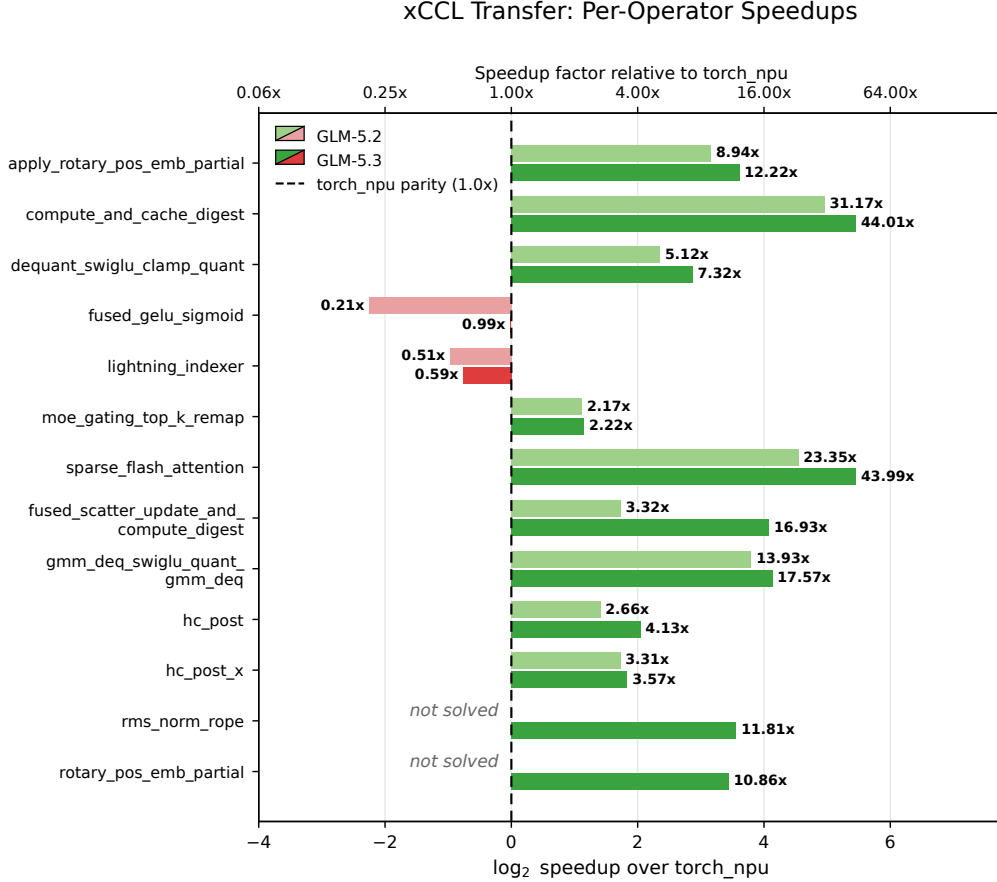


Figure 7: Per-operator speedups relative to `torch_npu` for xCCL tasks solved by GLM-5.2 or GLM-5.3. Paired bars compare the two backbones on the 11 commonly solved operators. The horizontal scale and color convention match Figure 6.

B Hyperparameter Configuration

Table 5 summarizes the main hyperparameters used by SAGE. Unless otherwise stated, the same configuration is used across all backbones and evaluation sets.

We use moderate negative credit, setting $\delta = \beta = 0.2$, so that a single failed episode or retrieved-but-unadopted item does not dominate accumulated positive evidence. Utility-aware reranking uses $\gamma = 0.3$, keeping semantic relevance as the primary retrieval signal while allowing historical utility to influence ranking. For consolidation, we set $K_{\text{ado}} = K_{\text{op}} = 3$, requiring repeated adoption across multiple operators before an experience becomes eligible for promotion. The resident hot memory is bounded by $B = 10\text{K}$ tokens.

Hyperparameters are selected once based on their intended numerical roles and preliminary system-level stability. The same configuration is used for Qwen3-Coder-Next, DeepSeek-V4-Flash, and GLM-5.2; no per-backbone or per-operator tuning is performed.

Table 5: Main hyperparameters used throughout SAGE.

Symbol	Value	Component	Description
T	30	Agent loop	Maximum interaction rounds
δ	0.2	ATU	Unsuccessful-episode penalty
β	0.2	ATU	Unadopted-retrieval penalty
$\eta_{\min}$	0.05	ATU	Minimum utility update rate
γ	0.3	Retrieval	Utility reranking strength
K_{ado}	3	UGC	Minimum adopted trajectories
K_{op}	3	UGC	Minimum distinct operators
B	10K tokens	UGC	Resident hot-memory budget
μ	0.1	UGC	Hot-memory usage EMA coefficient

C System Implementation and Evaluation Protocol

C.1 Utility-Gated Consolidation Procedure

Algorithm 1 details the budgeted replacement step of UGC, which is executed after each task-solving episode. UGC pools the current hot-memory items $\mathcal{H}$ with validated candidates $\mathcal{V}$ that satisfy the admission gates, ranks them by expected utility per resident token $U(m)$, and reconstructs the hot region under budget B . An item that does not fit is skipped while the scan continues, allowing shorter lower-ranked items to be admitted. Newly admitted items become consolidated, whereas displaced residents revert to validated status and regain retrieval eligibility.

Algorithm 1 Budgeted Hot-Memory Replacement

Require: Current hot memory $\mathcal{H}$, validated candidates $\mathcal{V}$, utility density $U(\cdot)$, token budget B

Ensure: Updated hot memory $\mathcal{H}'$

```

1:  $\mathcal{P} \leftarrow \mathcal{H} \cup \mathcal{V}$ 
2:  $\mathcal{H}' \leftarrow \emptyset$ 
3: for all  $m \in \mathcal{P}$  sorted by decreasing  $U(m)$  do
4:   if  $\text{Tok}(\mathcal{H}') + L_m \leq B$  then
5:      $\mathcal{H}' \leftarrow \mathcal{H}' \cup m$ 
6:   end if
7: end for
8:  $\sigma_m \leftarrow \text{consolidated}, \forall m \in \mathcal{H}' \setminus \mathcal{H}$ 
9:  $\sigma_m \leftarrow \text{validated}, \forall m \in \mathcal{H} \setminus \mathcal{H}'$ 
10: return  $\mathcal{H}'$ 

```

C.2 Infrastructure and Agent Runtime

Execution environment. Experiments are executed on Ascend 910C infrastructure with 8 accelerator cards (16 accelerator dies in total) per server. Multiple kernel-generation workers run concurrently, with each agent worker assigned one accelerator card. The host environment is Huawei Cloud EulerOS 2.0 on ARM64. The core software stack is:

- **NPU runtime:** CANN 8.5.1 with driver 25.5.1;
- **Python stack:** Python 3.11.14, `torch_npu` 2.9.0, and `vllm-ascend` 0.18.0rc1;
- **Model serving:** all evaluated backbones are deployed locally through `vllm-ascend`. DeepSeek-V4-Flash uses DP= 4/TP= 4, while GLM-5.2 and GLM-5.3 use TP= 16; both enable expert parallelism and Ascend-native quantization. Qwen3-Coder-Next follows the official `vllm-ascend` deployment recipe.

Unspecified serving parameters follow the corresponding `vllm-ascend` defaults. The agent runtime is implemented through OpenCode skills, which orchestrate the iterative generation, compilation, execution, diagnosis, and memory-update loop; the evaluated local backbone provides the kernel-generation and reasoning capability.

Device-level diagnostics. The agent can autonomously invoke both lightweight AscendC probes and CANN-native debugging/profiling utilities:

- **In-kernel probes:** `printf`, `DumpTensor`, and `DumpAccChkPoint`;
- **Runtime debugging:** `msDebug` and `msSanitizer` for memory violations, race conditions, synchronization failures, and device-side faults;
- **Performance diagnosis:** `msProf` and `torch_npu.profiler` for kernel-level execution and bottleneck analysis.

These tools are exposed to the agent as callable diagnostics rather than manually injected feedback, enabling tool-augmented localization of runtime and performance failures during refinement.

C.3 Initialization and Experimental Integrity

Cold-start knowledge construction. SAGE initializes architecture-specific knowledge through document distillation and differential mining. We distill and manually consolidate official AscendC API documentation, operator-development tool user guides, and AscendC best practices into three lightweight references: (i) a compact AscendC quick reference, (ii) a debugging guide, and (iii) an `add_custom` implementation example. Conditioned on these references, the agent generates kernels for simple Level-1 and Level-2 NPUKernelBench operators. Once compilation succeeds, we compare the generated kernel with an expert solution to extract reusable implementation patterns for cold retrieval memory. The operators used for differential mining are strictly disjoint from the 88 evaluation operators.

Controlled comparison and evaluation protocol. All memory-based methods start from an identical initial memory state. Refinement, Static RAG, Value Memory, and SAGE share the same backbone, agent harness, prompt structure, diagnostic tool access, interaction budget ($T = 30$), and evaluation harness; only the memory mechanism differs. The 88 evaluation operators form the learning stream, with each result recorded before any corresponding memory updates for subsequent operators. Cross-backbone transfer evaluates the same operators using fixed, previously evolved memories, without further memory updates.

Anti-hacking safeguards. During task solving on evaluation operators, an autonomous coding agent may exploit implementation shortcuts that inflate apparent correctness or speedup without reflecting genuine kernel-synthesis capability. We therefore enforce the following evaluation-integrity constraints for all methods:

- **No equivalent high-level substitution.** The agent may not implement the target computation by directly invoking a higher-level AscendC primitive that subsumes its core functionality (e.g., using `AscendC::Sigmoid` as the core implementation of `Swish`).
- **No expert-implementation leakage.** The agent may not inspect or search expert or structurally similar operator implementations outside the designated example directory.
- **No semantic shortcuts.** Optimizations may not remove required functionality, omit boundary handling, or hard-code tiling parameters for specific evaluation inputs.
- **No invalid precision trade-offs.** Reduced-precision transformations are permitted only when the resulting kernel satisfies the same correctness criteria used for evaluation.

C.4 Correctness and Performance Measurement

Correctness validation. For NPUKernelBench operators, generated kernels are validated against the corresponding `torch_npu` implementation using the benchmark error metrics, including average/maximum absolute error, average/-maximum relative error, and average accuracy. A kernel is considered correct when

$$\max_abs_error \leq 10^{-3}, \quad \max_rel_error \leq 10^{-3}. \quad (13)$$

Latency measurement. Latency is measured using device-side Ascend NPU events to avoid host-side scheduling and synchronization overhead in wall-clock timing. We instantiate the event timer with `torch.npu.Event(enable_timing=True)` and apply the same measurement protocol to every generated kernel and its `torch_npu` reference:

- execute 5 warm-up iterations;
- synchronize the NPU and record the start event;
- execute 100 timed iterations;
- record the end event, synchronize, and compute elapsed event time.

We report the mean per-execution latency over the 100 timed iterations. Generated kernels and their `torch_npu` references use identical inputs, warm-up counts, repetition counts, and synchronization boundaries. For each operator,

$$T_{\text{best}} = \min_{k \in \mathcal{K}_{\text{correct}}} T(k),$$

where $\mathcal{K}_{\text{correct}}$ denotes all correctness-passing candidates produced within the interaction budget.

D Additional Analyses and Controlled Experiments

D.1 Expert Audit of Adoption Records

Audit scope. We examine whether recorded adoption corresponds to observable code revisions. We randomly sample 50 records from all records labeled adopted in the saved generation traces. The sampling unit is an adoption record.

Verification. For each sampled record, an Ascend C expert among the authors checks the associated experience and adoption rationale against the corresponding code revision, assigning one of three correspondence ratings: *full*, *partial*, or *unrelated*. The saved trajectory provides the code evidence for this assessment.

Result and interpretation. The expert rated 44 records (88%) as full, six (12%) as partial, and none as unrelated (Table 6). Every inspected record thus showed at least partial correspondence with code revisions. These findings support recorded adoption as a trace-grounded proxy for actual memory use, providing an empirical basis for adoption-aware credit assignment in ATU.

Table 6: Expert-rated correspondence between adoption and code revisions for 50 records randomly sampled from all trace records labeled adopted.

Correspondence	Count	Percentage
Full	44	88%
Partial	6	12%
Unrelated	0	0%

D.2 Fixed-Bank Credit Assignment

The end-to-end ablation in Section 4.6 changes the memory accumulated over time together with its utility estimates. To isolate credit quality itself, we freeze an identical final SAGE memory bank, including its cold and hot contents, and replay the same historical adoption traces to construct three cold-memory ranking variants: relevance-only retrieval ($\gamma = 0$), uniform credit that assigns the terminal score to every retrieved item, and ATU. Hot memory remains identical across all variants, so only cold-memory utility reranking differs.

Table 7: Credit-assignment control under a shared frozen memory bank. All rates are percentages.

Ranking signal	ER	L3 ER	Fast _{1.0}
Relevance Only	84.1	61.1	70.3
Uniform Credit	89.8	72.2	79.7
ATU	95.5	88.9	86.9

Under the same memory contents, ATU consistently improves both correctness and optimization over uniform credit and relevance-only ranking. ATU consistently outperforms both controls across all reported metrics, supporting the intended role of adoption tracing: verifier outcomes provide aggregate supervision over generated kernels, whereas ATU more precisely attributes this evidence to the retrieved experiences actually used during synthesis.

E Formal Properties of ATU and UGC

We establish several properties of ATU and UGC that motivate their design choices in persistent kernel synthesis. The analysis focuses on bounded credit, adaptive utility estimation, controlled utility-aware retrieval, and usage-aware consolidation.

E.1 Property 1: Bounded and normalized adoption credit.

Recall that the terminal episode score satisfies $z \in [-\delta, 1]$, and ATU assigns

$$\xi_m = \begin{cases} z/|\mathcal{A}|, & m \in \mathcal{A}, \\ -\beta, & m \in \mathcal{R} \setminus \mathcal{A}, \end{cases} \quad (14)$$

where $\mathcal{R}$ and $\mathcal{A}$ are the episode-deduplicated retrieved and adopted sets, respectively. Let $q = \max\{\delta, \beta\}$ and assume $0 \leq \delta, \beta \leq 1$. Then

$$\xi_m \in [-q, 1], \quad \sum_{m \in \mathcal{A}} \xi_m = z \quad (\mathcal{A} \neq \emptyset). \quad (15)$$

Proof. For $m \in \mathcal{A}$, $|\mathcal{A}| \geq 1$ and $z/|\mathcal{A}| \in [-\delta, 1]$; otherwise $\xi_m = -\beta$, proving the first claim. For the second, $\sum_{m \in \mathcal{A}} z/|\mathcal{A}| = z$. $\square$

Thus, the total credit assigned to adopted experiences equals the terminal episode score regardless of how many experiences are adopted. This normalization prevents larger adoption sets from receiving more total credit. Moreover, moderate δ and β are appropriate for kernel synthesis: an unsuccessful candidate does not imply that every adopted hardware insight was harmful. With our default $\delta = \beta = 0.2$, negative item-level credit is lower bounded by -0.2 .

E.2 Property 2: Stable but continually adaptive utility.

ATU updates each item according to

$$u_m \leftarrow (1 - \eta_m)u_m + \eta_m \xi_m, \quad \eta_m = \max \left\{ \frac{1}{1 + n_m^{\text{ret}}}, \eta_{\min} \right\}, \quad (16)$$

with $u_m^{(0)} = 0$ and $0 < \eta_{\min} \leq 1$. If $\xi_m \in [-q, 1]$, then

$$u_m^{(t)} \in [-q, 1] \quad \forall t. \quad (17)$$

Proof. Since $0 < \eta_m \leq 1$, Eq. (16) is a convex combination of u_m and ξ_m . The interval $[-q, 1]$ is convex and contains the initial value, so the result follows by induction. $\square$

The reciprocal term progressively stabilizes an item as retrieval evidence accumulates, while the nonzero floor prevents its utility from becoming permanently frozen. Starting from the utility estimate at update t , after h subsequent updates, the weight retained from $u_m^{(t)}$ is at most

$$\prod_{j=0}^{h-1} (1 - \eta_{m,t+j}) \leq (1 - \eta_{\min})^h. \quad (18)$$

Hence old evidence is forgotten geometrically. For $\eta_{\min} = 0.05$, this bound corresponds to a half-life of approximately 13.5 updates. This behavior is intentional: unlike stationary bandit estimation, the utility of kernel experience can change as the task stream, available memory, and generator context evolve. ATU therefore tracks retrieval-conditioned utility rather than requiring convergence to a fixed stationary value.

E.3 Property 3: Controlled utility-aware reranking.

Historical utility should help distinguish among semantically plausible experiences, but should not make a previously useful yet task-irrelevant item dominate retrieval. SAGE therefore uses utility only as a multiplicative modulation of the relevance score:

$$s(m) = s_{\text{rel}}(m)(1 + \gamma u_m), \quad s_{\text{rel}}(m) \geq 0, \quad 0 \leq \gamma < 1. \quad (19)$$

From Property E.2, $u_m \in [-q, 1]$, where $q = \max\{\delta, \beta\} \leq 1$. It follows directly that

$$1 - \gamma q \leq 1 + \gamma u_m \leq 1 + \gamma, \quad (20)$$

so the modulation factor remains strictly positive.

We next characterize when utility can change the ordering between two candidates. Consider m_1 and m_2 with positive relevance scores, and suppose m_1 is semantically more relevant, $s_{\text{rel}}(m_1) > s_{\text{rel}}(m_2)$. The most adversarial utility

configuration for preserving this ordering assigns the minimum possible utility to m_1 and the maximum possible utility to m_2 , i.e., $u_{m_1} = -q$ and $u_{m_2} = 1$. Therefore,

$$s(m_1) \geq s_{\text{rel}}(m_1)(1 - \gamma q), \quad s(m_2) \leq s_{\text{rel}}(m_2)(1 + \gamma). \quad (21)$$

Hence a sufficient condition for the relevance ordering to remain unchanged under *any* admissible utility values is

$$\frac{s_{\text{rel}}(m_1)}{s_{\text{rel}}(m_2)} > \frac{1 + \gamma}{1 - \gamma q} \implies s(m_1) > s(m_2). \quad (22)$$

With our defaults $q = 0.2$ and $\gamma = 0.3$,

$$1 + \gamma u_m \in [0.94, 1.30], \quad \frac{1 + \gamma}{1 - \gamma q} \approx 1.38. \quad (23)$$

Thus utility can reorder candidates with comparable semantic relevance, while sufficiently large relevance gaps remain preserved. The asymmetry is intentional: positive utility reflects successful adoption-traced evidence, whereas negative evidence is treated conservatively because non-adoption or episode failure does not necessarily imply harmful knowledge. This is particularly appropriate for NPU kernel synthesis, where a historically useful experience may still be inapplicable to a structurally different operator.

E.4 Property 4: Resident priority decays under persistent non-use.

UGC updates the usage estimate of each resident item as

$$\hat{p}_m \leftarrow (1 - \mu)\hat{p}_m + \mu \mathbf{1}_{m \text{ is used}}, \quad 0 < \mu < 1, \quad (24)$$

and ranks resident knowledge by

$$U(m) = \frac{u_m \hat{p}_m}{L_m}. \quad (25)$$

For analysis, let $\hat{p}_m^{(t)}$ denote the usage estimate after the t -th memory update. If item m is not used during the next h updates, Eq. (24) recursively gives

$$\hat{p}_m^{(t+h)} = (1 - \mu)^h \hat{p}_m^{(t)}. \quad (26)$$

If u_m and L_m remain unchanged over this interval, its resident priority decays at the same rate:

$$U_{t+h}(m) = (1 - \mu)^h U_t(m). \quad (27)$$

The decay rate has a direct temporal interpretation. The number of consecutive unused updates required to halve the usage contribution is

$$h_{1/2} = \frac{\log(1/2)}{\log(1 - \mu)}. \quad (28)$$

With our default $\mu = 0.1$, $h_{1/2} \approx 6.6$ memory updates. Thus recently useful knowledge is retained smoothly, while persistently unused knowledge loses resident priority rather than remaining permanently consolidated.

More importantly, this decay guarantees finite-time loss of priority relative to any persistently useful alternative. Consider a stale resident item m_a with $U_t(m_a) > 0$, and suppose there exists an alternative candidate m_b whose priority satisfies

$$U_{t+j}(m_b) \geq c > 0 \quad \text{for all } j \geq 0,$$

where $c < U_t(m_a)$. If m_a remains unused and its utility and token cost remain unchanged, then

$$U_{t+h}(m_a) = (1 - \mu)^h U_t(m_a) < c \quad (29)$$

for any integer h satisfying

$$h > \frac{\log(U_t(m_a)/c)}{-\log(1 - \mu)}. \quad (30)$$

Since $0 < 1 - \mu < 1$, the right-hand side is finite. Hence a persistently useful candidate eventually outranks any resident item that remains unused.

This property is central to UGC: consolidation is not irreversible promotion. In persistent NPU kernel synthesis, knowledge useful for earlier operators may become less relevant as computation patterns, tensor layouts, or optimization regimes change. Usage-aware decay retains recurrently useful knowledge while allowing stale expertise to lose resident priority.

F Qualitative Examples of Memory Evolution and Kernel Synthesis

This appendix complements the aggregate evaluation with concrete examples of how SAGE selects retrieved experience, assigns post-episode credit, represents evolving memory, and produces complete Ascend C kernels. We first examine two task-solving trajectories that expose the distinction between retrieval and actual adoption. We then document the serialized memory schemas and provide a representative fused kernel generated by SAGE, together with the task-level instruction used to invoke the skill-based agent.

F.1 Adoption-Aware Memory Use

Diagnosing a host–device boundary error in GatherV3. At iteration 9, GatherV3 failed during correctness validation with a segmentation fault. The preceding revision had removed `ValueDepend` from the scalar axis input, but the host-side tiling function still dereferenced the data pointer returned by `GetData`. Without `ValueDepend`, this call exposes an NPU device address that cannot be dereferenced by the host CPU. Table 8 summarizes the five retrieved experiences and the agent’s adoption decisions.

Table 8: Retrieved experiences for the GatherV3 correctness-stage runtime failure. Relevance is the retriever score; adoption is determined from the subsequent diagnosis and code revision.

ID	Retrieved experience	Rel.	Adopted
181	Host-side tiling cannot dereference device-resident tensor data returned by <code>GetData</code> .	69.17	Yes
193	A kernel entry can read a scalar directly from GM through a <code>__gm__</code> pointer cast.	67.42	Yes
61	Scalar inputs can be exposed to tiling through <code>ValueDepend (REQUIRED)</code> and passed in <code>TilingData</code> .	57.43	No
139	Nested tiling structures require <code>GET_TILING_DATA_WITH_STRUCT</code> .	49.22	No
159	<code>TilingData</code> field types must match their kernel-side consumers.	32.47	No

The agent adopted items 181 and 193: the former identified the failure mechanism, whereas the latter supplied the applicable repair. Guided by the adopted pair, the agent removed all axis-dependent computation from the host tiling function, retained only input dimensions in `TilingData`, and moved the scalar read and dependent work partitioning into the device side:

```
int64_t axisVal = *reinterpret_cast<__gm__ int64_t*>(axis);
if (axisVal < 0)
    axisVal += static_cast<int64_t>(dimNum);
uint32_t axis = static_cast<uint32_t>(axisVal);
this->srcDimSize = xDims[axis];

uint32_t outerSize = 1;
for (uint32_t d = 0; d < axis; ++d)
    outerSize *= xDims[d];
uint32_t innerSize = 1;
for (uint32_t d = axis + 1; d < dimNum; ++d)
    innerSize *= xDims[d];
```

This example illustrates why relevance alone is insufficient for memory credit: all five items were retrieved for the same failure, yet only two were causally reflected in the repair.

Kernel synthesis trajectories of Mla. The Level-3 Mla (Multi-head Latent Attention) operator of `NPUKernelBench` fuses two QK paths, softmax, RoPE, and PV matrix multiplication. SAGE required 23 generation attempts to obtain the first correct kernel and then used the remaining budget for optimization. Table 9 condenses the principal state transitions.

F.2 Memory Record Schemas

Tables 10 and 11 document SAGE’s item records and hot-memory snapshots. Each item record stores a retrieval-facing rule, diagnostic evidence, ATU utility, retrieval and adoption counts, cross-operator adoption history, and lifecycle state. These fields remain in the cold store after promotion, preserving the item’s history. The Example values are synthetic and illustrate the schemas only.

Table 9: Key transitions in the Mla task-solving trajectory on ascend910_93.

Iteration	Diagnosis	Revision	Outcome
0–2	A shared Matmul object changed its K dimension despite fixed tiling; the default task type also incorrectly assigned Cube work on AIV core.	Separate the QK and PV configurations and declare <code>KERNEL_TYPE_MIX_AIC_1_0</code> .	Compilation succeeds; Matmul no longer hangs.
10–13	Multi-token execution exposed unmatched AIC–AIV synchronization.	Isolate a single-token path, then pair cross-core events within the token loop.	The hang becomes a diagnosable numerical mismatch.
18–23	<code>DumpTensor</code> showed zero softmax inputs; four host inputs were non-contiguous, and the QK path omitted the RoPE dimensions.	Synchronize the asynchronous Matmul write, apply <code>AutoContiguous()</code> , and materialize the full 576-dimensional Q/K representations.	First correct kernel that provides optimization baseline: 0.420 ms ($1.91\times$).
24–26	Double buffering violated the MTE3 handoff; a global barrier was also overly conservative.	Revert the unsafe buffer change and replace <code>PIPE_ALL</code> with an MTE3–MTE2 event.	$1.95\times$.
27	Work remained single-core.	Partition query tokens across cores.	$3.71\times$.
29	A relay copy materialized contiguous QK data before PV.	Split PV into nope and RoPE Matmuls that read the original GM tensors directly.	0.155 ms ; $5.13\times$.

A hot-memory snapshot identifies the items selected under a resident token budget. Before promotion, access frequency is estimated from retrievals over eligible episodes. Resident items then bypass retrieval, and their access estimates are updated from observed context usage. Their utility per token, $U(m) = u_m \hat{p}_m / L_m$, combines estimated benefit, access frequency, and context cost.

Table 10: Schema of a memory-item record.

Field	Type	Description	Example
<code>id</code>	integer	Stable memory-item identifier.	10035
<code>title</code>	string	Compact retrieval-facing rule.	Reduce algorithmic complexity and branches
<code>type</code>	enum	Experience type used for filtering.	<code>algorithmic_optimization</code>
<code>category</code>	string	Operator or failure category.	<code>Scan/Reduction</code>
<code>error_message</code>	string	Failure or inefficiency that motivated the item.	Quadratic polynomial evaluation with redundant writes
<code>code_diff</code>	object	Minimal incorrect/correct code pair.	<code>{wrong_code, correct_code}</code>
<code>summary</code>	string	Compressed explanation stored for retrieval.	Replace repeated powers with Horner evaluation
<code>u_m</code>	float	ATU utility estimate.	0.42
<code>n_ret</code>	integer	Number of retrievals.	6
<code>n_ado</code>	integer	Number of observed adoptions.	4
<code>adopted_operators</code>	list[string]	Distinct operators providing adoption evidence.	<code>[GroupGemm, ...]</code>
<code>sigma</code>	enum	Lifecycle state.	<code>consolidated</code>
<code>L_m</code>	integer	Resident token cost.	96
<code>hot_region</code>	boolean	Whether the item currently resides in hot memory.	<code>true</code>
<code>hot_last_used_episode</code>	integer/null	Most recent resident-context use.	18
<code>p_hat</code>	float	Estimated access frequency; updated by usage EMA after promotion.	0.35
<code>n_elig</code>	integer	Episodes for which the cold item was retrieval-eligible.	18
<code>density</code>	float	Expected utility per resident token, $U(m)$.	0.00153

Table 11: Schema of a hot-memory snapshot and its nested entry representation.

Field	Type	Description	Example
session_id	integer	Memory-evolution session identifier.	1
created_at	timestamp	Snapshot creation time.	20260811_111049
budget	integer	Maximum resident-context tokens.	10000
n_ops	integer	Completed operator trajectories at snapshot time.	18
hot_entries	list[object]	Items selected under the token budget.	[[id: 238, ...]]
<i>Nested hot-entry fields</i>			
id	integer	Identifier of the resident item.	238
title	string	Consolidated rule inserted into persistent context.	Use int32 generation plus an int64 cast for ArithProgression
c_tokens	integer	Tokens consumed by the consolidated representation.	61
u	float	Current ATU utility estimate.	0.62
p_hat	float	Resident-context usage estimate.	0.44
density	float	Ranking score $U(m) = u_m \hat{p}_m / L_m$.	0.00447

F.3 Generated Kernel and Skill-Based Task Instruction

SAGE is implemented through executable skills rather than a monolithic handcrafted prompt. The generator skill constructs and validates a complete Ascend C operator project; after correctness is reached, the optimizer skill continues from the best feasible kernel using on-device performance feedback. The task-level instruction used across experiments follows the template below.

Use the ascend-kernel-generator skill to generate the {operator_category} operator {operator_name} until it passes compilation and correctness tests, and then use the ascend-kernel-optimizer skill for continual optimization. You may execute at most 30 iterations in total; once this budget is reached, stop immediately regardless of progress. Follow the complete skill workflows and do not use any hacking shortcuts. Code references are restricted to the skill’s own examples directory, and examples for other operators must not be inspected. When an error occurs, proactively apply the debugging techniques provided in the skill references to localize the faulty code. The local SocVersion is ascend910_93.

We present the sparse flash attention kernel generated by SAGE using GLM-5.3 as its backbone during the xCCL evaluation. It passed correctness validation and achieved a $43.99\times$ speedup over the reference torch_npu implementation. The file-specific background colors distinguish host registration and tiling (blue), the serialized tiling definition (yellow), and device-kernel code (green).

op_host/sparse_flash_attention.cpp

```

1 #include "sparse_flash_attention_tiling.h"
2 #include "register/op_def_registry.h"
3 #include "tiling/platform/platform_ascendc.h"
4 #include "tiling/tiling_api.h"
5
6 using namespace matmul_tiling;
7
8 namespace optiling {
9 // kernel-side task granularity: gSize q-heads joined (each q-head -> 1 row batch
10 // of gSize contiguous q rows sharing the same sparse selection).
11 static constexpr int32_t S1_TILE = 1;
12 static constexpr int32_t BASE_M = 16;
13 static constexpr int32_t BASE_N = 16;
14 static constexpr int32_t BASE_K = 64;
15 // kernel-side select-loop granularity (queries per select loop, <= S1_TILE rows
16 // are re-selected for each new query row; kept a host constant so kernel and
17 // tiling agree).
18 static constexpr int32_t SEL_ROWS = 1;
19 static constexpr int32_t N_BATCH = 32;
20 // MLA: 512 nope dims + 64 rope dims; output carries nope cols only.
21 static constexpr int32_t NOPE_DIM = 512;
22
23 static ge::graphStatus TilingFunc(gert::TilingContext *context)
24 {

```

```

25 auto ascendcPlatform = platform_ascendc::PlatformAscendC(context->GetPlatformInfo());
26 auto qShape = context->GetInputShape(0)->GetStorageShape();
27 uint32_t B = static_cast<uint32_t>(qShape.GetDim(0));
28 uint32_t S1 = static_cast<uint32_t>(qShape.GetDim(1));
29 uint32_t N1 = static_cast<uint32_t>(qShape.GetDim(2));
30
31 auto kShape = context->GetInputShape(1)->GetStorageShape();
32 // key layout: PA_BSND [blockNum, blockSize, N2, D]
33 uint32_t P = static_cast<uint32_t>(kShape.GetDim(1));
34 uint32_t N2 = static_cast<uint32_t>(kShape.GetDim(2));
35
36 auto idxShape = context->GetInputShape(3)->GetStorageShape();
37 // sparse_indices: [B, N2, S1, K]
38 uint32_t K = static_cast<uint32_t>(idxShape.GetDim(3));
39
40 uint32_t gSize = (N2 == 0) ? N1 : N1 / N2;
41
42 // head-group split: gSize % HG == 0 -> tasks = B*S1*(gSize/HG); each task
43 // runs the full gather+bmm1+softmax+bmm2 chain for HG joined rows of one
44 // (b, s1), enabling multi-core across the head dimension.
45 uint32_t HG = 16; // 8 and 16 both pass with relays fixed (same 0.106ms);
46 // keep 16: fewer tasks, larger bmm M per task.
47 while (HG > 1 && (gSize % HG != 0)) {
48     HG /= 2;
49 }
50 uint32_t nGroups = gSize / HG;
51
52 uint32_t aicNum = ascendcPlatform.GetCoreNumAic();
53 // one task = one (b, s1, headGroup): total joined-row batches to schedule
54 uint32_t totalTasks = B * S1 * nGroups;
55 uint32_t usedCoreNum = (totalTasks < aicNum) ? totalTasks : aicNum;
56 if (usedCoreNum == 0) {
57     usedCoreNum = 1;
58 }
59
60 uint64_t ubSizePlat = 0;
61 ascendcPlatform.GetCoreMemSize(platform_ascendc::CoreMemType::UB, ubSizePlat);
62 uint64_t l1SizePlat = 0;
63 ascendcPlatform.GetCoreMemSize(platform_ascendc::CoreMemType::L1, l1SizePlat);
64 int32_t l1Size = static_cast<int32_t>(l1SizePlat);
65
66 SparseFlashAttentionTilingData tiling;
67 tiling.set_batch(B);
68 tiling.set_seqQ(S1);
69 tiling.set_numHeadsQ(N1);
70 tiling.set_blockSize(P);
71 tiling.set_sparseBlkSize(P); // sparse block granularity == page granularity
72 tiling.set_sparseBlkCount(K);
73 tiling.set_gSize(gSize);
74 tiling.set_hgSplit(HG);
75 tiling.set_nIdxSize((K + N_BATCH - 1) / N_BATCH);
76
77 auto attrs = context->GetAttrs();
78 float scaleValue = 1.0f;
79 if (attrs != nullptr) {
80     const float *scalePtr = attrs->GetFloat(0);
81     if (scalePtr != nullptr) {
82         scaleValue = *scalePtr;
83     }
84 }
85 tiling.set_scaleValue(scaleValue);
86
87 uint32_t S2s = K * P; // selected KV span (columns of score matrix)
88
89 // bmm1: [HG, S2s, 576] A=Qjoined(BSND row-major, non-trans)
90 // B=KVgathered^T (gathered KV cache laid out [S2s, 576] rows, transposed)
91 MultiCoreMatmulTiling cubeTiling1(ascendcPlatform);
92 cubeTiling1.SetDim(1);
93 cubeTiling1.SetAType(TPosition::GM, CubeFormat::ND, matmul_tiling::DataType::DT_BF16, false);
94 cubeTiling1.SetBType(TPosition::GM, CubeFormat::ND, matmul_tiling::DataType::DT_BF16, true);
95 cubeTiling1.SetCType(TPosition::GM, CubeFormat::ND, matmul_tiling::DataType::DT_FLOAT);
96 cubeTiling1.SetShape(HG, S2s, 576);
97 cubeTiling1.SetOrgShape(gSize, S2s, 576);
98 cubeTiling1.SetFixSplit(BASE_M, BASE_N, BASE_K);
99 cubeTiling1.SetBias(false);
100 cubeTiling1.SetBufferSpace(l1Size, -1, -1);
101 if (cubeTiling1.GetTiling(tiling.cubeTiling1) == -1) {
102     return ge::GRAPH_FAILED;
103 }
104
105 // bmm2: [HG, 512, S2s] A=P(joined row-major), B=Vgathered nope cols

```

```

106 // (non-trans); C width 512 matches the [B,S1,N1,512] output row stride.
107 MultiCoreMatmulTiling cubeTiling2(ascendcPlatform);
108 cubeTiling2.SetDim(1);
109 cubeTiling2.SetAType(TPosition::GM, CubeFormat::ND, matmul_tiling::DataType::DT_BF16, false);
110 cubeTiling2.SetBType(TPosition::GM, CubeFormat::ND, matmul_tiling::DataType::DT_BF16, false);
111 cubeTiling2.SetCType(TPosition::GM, CubeFormat::ND, matmul_tiling::DataType::DT_BF16);
112 cubeTiling2.SetShape(HG, NOPE_DIM, S2s);
113 cubeTiling2.SetOrgShape(gSize, NOPE_DIM, S2s);
114 cubeTiling2.SetFixSplit(BASE_M, BASE_N, BASE_K);
115 cubeTiling2.SetBias(false);
116 cubeTiling2.SetBufferSpace(l1Size, -1, -1);
117 if (cubeTiling2.GetTiling(tiling.cubeTiling2) == -1) {
118     return ge::GRAPH_FAILED;
119 }
120
121 // softmax over S2s: src [SOFTMAX_M, S2s] half
122 constexpr uint32_t SOFTMAX_M = 4;
123 std::vector<int64_t> shapeVec = {SOFTMAX_M, static_cast<int64_t>(S2s)};
124 ge::Shape softmaxSrcShape(shapeVec);
125 uint32_t softmaxMinTmp = AscendC::GetSoftMaxMinTmpSize(softmaxSrcShape, sizeof(float), false);
126 uint32_t softmaxMaxTmp = AscendC::GetSoftMaxMaxTmpSize(softmaxSrcShape, sizeof(float), false);
127 uint32_t localWorkspaceSize = softmaxMaxTmp;
128 AscendC::SoftMaxTilingFunc(softmaxSrcShape, sizeof(float), localWorkspaceSize, tiling.softmaxTiling);
129 (void)softmaxMinTmp;
130
131 tiling.set_usedCoreNum(usedCoreNum);
132 tiling.set_blockDim(usedCoreNum);
133 tiling.SaveToBuffer(context->GetRawTilingData()->GetData(), context->GetRawTilingData()->GetCapacity());
134 context->GetRawTilingData()->SetDataSize(tiling.GetDataSize());
135
136 context->SetBlockDim(usedCoreNum);
137
138 // workspace: sys part + 16MB reserved gap + user part
139 // user per block: score [g, S2s] float | P [g, S2s] half | KVg [S2s,576] bf16
140 // | Vg [S2s,512] bf16 | Qj [g,576] bf16
141 constexpr uint64_t RESERVED_WS = 16ULL * 1024 * 1024;
142 uint64_t scoreSlot = static_cast<uint64_t>(HG) * S2s * sizeof(float);
143 uint64_t pSlot = static_cast<uint64_t>(HG) * S2s * sizeof(half);
144 uint64_t kvSlot = static_cast<uint64_t>(S2s) * 576 * 2;
145 uint64_t vgSlot = static_cast<uint64_t>(S2s) * NOPE_DIM * 2;
146 uint64_t qjSlot = static_cast<uint64_t>(HG) * 576 * 2;
147 uint64_t userWorkspaceSize = (scoreSlot + pSlot + kvSlot + vgSlot + qjSlot) * usedCoreNum;
148 size_t systemWorkspaceSize = static_cast<size_t>(ascendcPlatform.GetLibApiWorkSpaceSize());
149 size_t *currentWorkspace = context->GetWorkspaceSizes(1);
150 currentWorkspace[0] = userWorkspaceSize + systemWorkspaceSize + RESERVED_WS;
151
152 return ge::GRAPH_SUCCESS;
153 }
154 } // namespace optiling
155
156 namespace ge {
157 static ge::graphStatus InferShape(gert::InferShapeContext *context)
158 {
159     const gert::Shape *queryShape = context->GetInputShape(0);
160     gert::Shape *attentionOutShape = context->GetOutputShape(0);
161     *attentionOutShape = *queryShape;
162     return ge::GRAPH_SUCCESS;
163 }
164
165 static ge::graphStatus InferDataType(gert::InferDataTypeContext *context)
166 {
167     context->SetOutputDataType(0, context->GetInputDataType(0));
168     return ge::GRAPH_SUCCESS;
169 }
170 } // namespace ge
171
172 namespace ops {
173 class SparseFlashAttention : public OpDef {
174 public:
175     explicit SparseFlashAttention(const char* name) : OpDef(name)
176     {
177         this->Input("query")
178             .ParamType(REQUIRED)
179             .DataType({ge::DT_BF16})
180             .Format({ge::FORMAT_ND})
181             .UnknownShapeFormat({ge::FORMAT_ND});
182         this->Input("key")
183             .ParamType(REQUIRED)
184             .DataType({ge::DT_BF16})
185             .Format({ge::FORMAT_ND})
186             .UnknownShapeFormat({ge::FORMAT_ND});

```



```

187     this->Input("value")
188         .ParamType(REQUIRED)
189         .DataType({ge::DT_BF16})
190         .Format({ge::FORMAT_ND})
191         .UnknownShapeFormat({ge::FORMAT_ND});
192     this->Input("sparse_indices")
193         .ParamType(REQUIRED)
194         .DataType({ge::DT_INT32})
195         .Format({ge::FORMAT_ND})
196         .UnknownShapeFormat({ge::FORMAT_ND});
197     this->Input("block_table").ParamType(OPTIONAL).DataType({ge::DT_INT32})
198         .Format({ge::FORMAT_ND}).UnknownShapeFormat({ge::FORMAT_ND});
199     this->Input("actual_seq_lengths_query").ParamType(OPTIONAL).DataType({ge::DT_INT32})
200         .Format({ge::FORMAT_ND}).UnknownShapeFormat({ge::FORMAT_ND});
201     this->Input("actual_seq_lengths_kv").ParamType(OPTIONAL).DataType({ge::DT_INT32})
202         .Format({ge::FORMAT_ND}).UnknownShapeFormat({ge::FORMAT_ND});
203     this->Input("query_rope").ParamType(OPTIONAL).DataType({ge::DT_BF16})
204         .Format({ge::FORMAT_ND}).UnknownShapeFormat({ge::FORMAT_ND});
205     this->Input("key_rope").ParamType(OPTIONAL).DataType({ge::DT_BF16})
206         .Format({ge::FORMAT_ND}).UnknownShapeFormat({ge::FORMAT_ND});
207     this->Output("attention_out")
208         .ParamType(REQUIRED)
209         .DataType({ge::DT_BF16})
210         .Format({ge::FORMAT_ND})
211         .UnknownShapeFormat({ge::FORMAT_ND});
212     this->Attr("scale_value").AttrType(OPTIONAL).Float(1.0);
213     this->Attr("sparse_block_size").AttrType(OPTIONAL).Int();
214     this->Attr("layout_query").AttrType(OPTIONAL).String("BSND");
215     this->Attr("layout_kv").AttrType(OPTIONAL).String("PA_BSND");
216     this->Attr("sparse_mode").AttrType(OPTIONAL).Int(3);
217
218     this->SetInferShape(ge::InferShape).SetInferDataType(ge::InferDataType);
219     this->AICore()
220         .SetTiling(optiling::TilingFunc);
221     this->AICore().AddConfig("ascend910_93")
222         .AddConfig("ascend910b");
223 }
224 };
225 OP_ADD(SparseFlashAttention);
226 } // namespace ops

```

op_host/sparse_flash_attention_tiling.h

```

1  #ifndef SPARSE_FLASH_ATTENTION_TILING_H
2  #define SPARSE_FLASH_ATTENTION_TILING_H
3
4  #include "register/tilingdata_base.h"
5  #include "tiling/tiling_api.h"
6
7  namespace optiling {
8  BEGIN_TILING_DATA_DEF(SparseFlashAttentionTilingData)
9  TILING_DATA_FIELD_DEF(uint32_t, batch); // B
10 TILING_DATA_FIELD_DEF(uint32_t, seqQ); // S1 rows of query per batch
11 TILING_DATA_FIELD_DEF(uint32_t, numHeadsQ); // N1
12 TILING_DATA_FIELD_DEF(uint32_t, blockSize); // KV page size P
13 TILING_DATA_FIELD_DEF(uint32_t, sparseBlkSize); // sparse block size (== blockSize in our tests)
14 TILING_DATA_FIELD_DEF(uint32_t, sparseBlkCount); // K = selected blocks per (b,hq,s)
15 TILING_DATA_FIELD_DEF(uint32_t, gSize); // N1 / N2 (N2 == 1)
16 TILING_DATA_FIELD_DEF(uint32_t, hgSplit); // q-heads per task group (gSize % hgSplit == 0)
17 TILING_DATA_FIELD_DEF(uint32_t, nIdxSize); // ceil(K / N_BATCH): per-select-loop UB rows
18 TILING_DATA_FIELD_DEF(uint32_t, usedCoreNum); // number of AIC:AIV blocks launched
19 TILING_DATA_FIELD_DEF(uint32_t, blockDim);
20 TILING_DATA_FIELD_DEF(float, scaleValue); // softmax scale
21 TILING_DATA_FIELD_DEF_STRUCT(TCubeTiling, cubeTiling1); // bmm1: [S1_TILE, S2s, 576]
22 TILING_DATA_FIELD_DEF_STRUCT(TCubeTiling, cubeTiling2); // bmm2: [S1_TILE, 576, S2s]
23 TILING_DATA_FIELD_DEF_STRUCT(SoftMaxTiling, softmaxTiling); // softmax over S2s, src half
24 END_TILING_DATA_DEF;
25
26 REGISTER_TILING_DATA_CLASS(SparseFlashAttention, SparseFlashAttentionTilingData)
27 } // namespace optiling
28
29 #endif

```

op_kernel/sparse_flash_attention.cpp

```

1  #include "kernel_operator.h"
2  #include "lib/matmul_intf.h"
3

```

```

4 using namespace AscendC;
5 using namespace matmul;
6
7 // MLA sparse attention decode: per query row, gather sparse-selected KV pages
8 // (nope 512 + rope 64 = 576 cols), bmm1(Qjoined x KVg~T) -> score float,
9 // softmax -> P half, bmm2(P x KVg) -> 0 bf16. KFC MIX AIC 1:2, AIV drives cube.
10 constexpr uint32_t SOFTMAX_ROWS = 4;
11 constexpr uint32_t FLOAT_NUM_PER_BLOCK = 8;
12 constexpr uint32_t NOPE_DIM = 512;
13 constexpr uint32_t ROPE_DIM = 64;
14 constexpr uint32_t DQK = NOPE_DIM + ROPE_DIM; // 576
15 constexpr uint32_t N_BATCH = 32; // sparse indices resolved per loop
16 constexpr uint32_t P_MAX = 16; // max KV page rows staged per relay
17
18 using AT1 = MatmulType<TPosition::GM, CubeFormat::ND, bfloat16_t, false>; // Qj [g, 576]
19 using BT1 = MatmulType<TPosition::GM, CubeFormat::ND, bfloat16_t, true>; // KVg [S2s, 576] trans
20 using CT1 = MatmulType<TPosition::GM, CubeFormat::ND, float>; // score float
21 using AT2 = MatmulType<TPosition::GM, CubeFormat::ND, bfloat16_t, false>; // P [g, S2s] bf16
22 using BT2 = MatmulType<TPosition::GM, CubeFormat::ND, bfloat16_t, false>; // Vg [S2s, 576]
23 using CT2 = MatmulType<TPosition::GM, CubeFormat::ND, bfloat16_t>; // out bf16
24
25 // REGIST_MATMUL_OBJ must stay in the extern-C entry scope (KfcServer/KfcCommClient
26 // locals die on scope exit, killing every AIC server -> later IterateAll hangs;
27 // see FlashAttentionScoreWithLargeHeadDim golden).
28 class SparseFlashAttentionKernel {
29 public:
30     __aicore__ inline SparseFlashAttentionKernel() {}
31
32     Matmul<AT1, BT1, CT1, CT1, CFG_NORM> bmm1;
33     Matmul<AT2, BT2, CT2, CT2, CFG_NORM> bmm2;
34     TPipe pipe_;
35     uint32_t btStride_ = 0; // block_table columns per batch (set by entry)
36
37     __aicore__ inline void Init(
38         GM_ADDR query, GM_ADDR key, GM_ADDR value, GM_ADDR sparse_indices,
39         GM_ADDR block_table, GM_ADDR actual_seq_lengths_query, GM_ADDR actual_seq_lengths_kv,
40         GM_ADDR query_rope, GM_ADDR key_rope, GM_ADDR attention_out,
41         GM_ADDR workspace, const SparseFlashAttentionTilingData &tilingData)
42     {
43         B_ = tilingData.batch;
44         S1_ = tilingData.seqQ;
45         N1_ = tilingData.numHeadsQ;
46         P_ = tilingData.blockSize;
47         K_ = tilingData.sparseBlkCount;
48         gSize_ = tilingData.gSize;
49         hg_ = tilingData.hgSplit;
50         nGroups_ = gSize_ / hg_;
51         scale_ = tilingData.scaleValue;
52         usedCoreNum_ = tilingData.usedCoreNum;
53         S2s_ = K_ * P_;
54         nIdxSize_ = tilingData.nIdxSize;
55         totalTasks_ = B_ * S1_ * nGroups_;
56
57         queryGm_.SetGlobalBuffer(reinterpret_cast<__gm__ bfloat16_t *>(query));
58         keyGm_.SetGlobalBuffer(reinterpret_cast<__gm__ bfloat16_t *>(key));
59         valueGm_.SetGlobalBuffer(reinterpret_cast<__gm__ bfloat16_t *>(value));
60         idxGm_.SetGlobalBuffer(reinterpret_cast<__gm__ int32_t *>(sparse_indices));
61         btGm_.SetGlobalBuffer(reinterpret_cast<__gm__ int32_t *>(block_table));
62         qRopeGm_.SetGlobalBuffer(reinterpret_cast<__gm__ bfloat16_t *>(query_rope));
63         kRopeGm_.SetGlobalBuffer(reinterpret_cast<__gm__ bfloat16_t *>(key_rope));
64         outGm_.SetGlobalBuffer(reinterpret_cast<__gm__ bfloat16_t *>(attention_out));
65
66         // user workspace per block: [score float | P bf16 | KVg [S2s,576] | Vg [S2s,512] | Qj [hg,576]]
67         scoreSlotElems_ = static_cast<uint64_t>(hg_) * S2s_;
68         pSlotElems_ = static_cast<uint64_t>(hg_) * S2s_;
69         kvSlotBytes_ = static_cast<uint64_t>(S2s_) * DQK * sizeof(bfloat16_t);
70         vgSlotBytes_ = static_cast<uint64_t>(S2s_) * NOPE_DIM * sizeof(bfloat16_t);
71         qjSlotBytes_ = static_cast<uint64_t>(hg_) * DQK * sizeof(bfloat16_t);
72         __gm__ uint8_t *userWs = GetUserWorkspace(workspace);
73         scoreWsGm_.SetGlobalBuffer(reinterpret_cast<__gm__ float *>(userWs));
74         pWsBase_ = userWs + scoreSlotElems_ * sizeof(float) * usedCoreNum_;
75         kvWsBase_ = pWsBase_ + pSlotElems_ * sizeof(bfloat16_t) * usedCoreNum_;
76         vgWsBase_ = kvWsBase_ + kvSlotBytes_ * usedCoreNum_;
77         qjWsBase_ = vgWsBase_ + vgSlotBytes_ * usedCoreNum_;
78
79         softmaxTiling_ = tilingData.softmaxTiling;
80
81         if (g_coreType == AIV) {
82             // softmax input rows: [SOFTMAX_ROWS, S2s] float; output P bf16
83             pipe_.InitBuffer(scoreTQue_, 1, SOFTMAX_ROWS * S2s_ * sizeof(float));
84             pipe_.InitBuffer(pTQue_, 1, SOFTMAX_ROWS * S2s_ * sizeof(bfloat16_t));

```

```

85     pipe_.InitBuffer(pHalfTBuf_, SOFTMAX_ROWS * S2s_ * sizeof(half));
86     pipe_.InitBuffer(pFloatTBuf_, SOFTMAX_ROWS * S2s_ * sizeof(float));
87     pipe_.InitBuffer(sumTBuf_, SOFTMAX_ROWS * FLOAT_NUM_PER_BLOCK * sizeof(float));
88     pipe_.InitBuffer(maxTBuf_, SOFTMAX_ROWS * FLOAT_NUM_PER_BLOCK * sizeof(float));
89     // sparse index staging + gather relay buffer (one KV page, P*512 bf16)
90     pipe_.InitBuffer(idxTQue_, 1, N_BATCH * sizeof(int32_t));
91     pipe_.InitBuffer(btTQue_, 1, N_BATCH * sizeof(int32_t)); // block_table row (K selects)
92     pipe_.InitBuffer(qgTQue_, 1, P_MAX * (NOPE_DIM + ROPE_DIM + NOPE_DIM) * sizeof(bfloat16_t));
93 }
94 }
95 // AIV sub0 drives gather + bmm1 + softmax + bmm2; sub1 helps softmax
96 __aicore__ inline void Process()
97 {
98     if ASCEND_IS_AIC
99         return;
100     uint32_t subIdx = static_cast<uint32_t>(GetBlockIdx() % GetSubBlockNum());
101     uint32_t blockIdx = static_cast<uint32_t>(GetBlockIdx() / GetSubBlockNum());
102     if (subIdx == 0) {
103         for (uint32_t task = blockIdx; task < totalTasks_; task += usedCoreNum_)
104             RunTask(task);
105         bmm1.End();
106         bmm2.End();
107     } else
108         return;
109 }
110 private:
111 // task = (b * S1 + s1) * nGroups + hg; hg-th head group of hg_ joined rows
112 __aicore__ inline void RunTask(uint32_t task)
113 {
114     uint32_t bs = task / nGroups_;
115     uint32_t hg = task % nGroups_;
116     uint32_t b = bs / S1_;
117     uint32_t s1 = bs % S1_;
118     uint32_t coreId = GetBlockIdx() / GetSubBlockNum();
119
120     GlobalTensor<float> scoreGm = scoreWsGm_[coreId * scoreSlotElems_];
121     GlobalTensor<bfloat16_t> kvGm;
122     kvGm.SetGlobalBuffer(reinterpret_cast<__gm__ bfloat16_t *>(kvWsBase_ + coreId * kvSlotBytes_));
123     GlobalTensor<bfloat16_t> vgGm;
124     vgGm.SetGlobalBuffer(reinterpret_cast<__gm__ bfloat16_t *>(vgWsBase_ + coreId * vgSlotBytes_));
125     GlobalTensor<bfloat16_t> qjGm;
126     qjGm.SetGlobalBuffer(reinterpret_cast<__gm__ bfloat16_t *>(qjWsBase_ + coreId * qjSlotBytes_));
127
128     GatherKV(b, s1, kvGm, vgGm); // KVg [S2s,576] (K nope+rope) + Vg [S2s,512]
129     BuildJoinedQ(b, s1, hg * hg_, qjGm); // Qj [hg_,576] for this head group
130     qjGm_ = qjGm;
131
132     // bmm1: Qj [HG,576] x KVg^T -> score float [HG, S2s]
133     bmm1.SetSingleShape(static_cast<int>(hg_), static_cast<int>(S2s_), static_cast<int>(DQK));
134     bmm1.SetTensorA(qjGm_);
135     bmm1.SetTensorB(kvGm, true);
136     bmm1.template IterateAll<true>(scoreGm, 0, false, false, false);
137
138     SoftmaxRows(task, 0);
139
140     // bmm2: P [HG,S2s] x Vg [S2s,512] -> 0 bf16 [HG,512].
141     // B is the gathered VALUE pages (a separate input tensor from key);
142     // C rows land on output heads [hg*HG, hg*HG+HG) with row stride 512.
143     GlobalTensor<bfloat16_t> pGm;
144     pGm.SetGlobalBuffer(reinterpret_cast<__gm__ bfloat16_t *>(
145         pWsBase_ + coreId * pSlotElems_ * sizeof(bfloat16_t)));
146     bmm2.SetSingleShape(static_cast<int>(hg_), static_cast<int>(NOPE_DIM), static_cast<int>(S2s_));
147     bmm2.SetTensorA(pGm);
148     bmm2.SetTensorB(vgGm);
149     GlobalTensor<bfloat16_t> oGm = outGm_[
150         (static_cast<uint64_t>(b) * S1_ + s1) * N1_ * NOPE_DIM +
151         static_cast<uint64_t>(hg) * hg_ * NOPE_DIM];
152     bmm2.template IterateAll<true>(oGm, 0, false, false, false);
153 }
154
155 // GM->GM gather relayed through UB (no GM->GM DataCopy(Pad) exists on A2).
156 // GM --MTE2--> UB (qgTQue_ VECIN) --MTE3--> GM using the same LocalTensor:
157 // UB2GM DataCopyPad accepts any LocalTensor whose HW position is UB, and
158 // VECIN maps to UB, so the dequeued VECIN tensor is a valid MTE3 source.
159 // Cross-pipeline order via SetFlag/WaitFlag(HardEvent::MTE2_MTE3).
160 // DataCopyExtParams.blockLen is BYTES; srcGap/dstGap are 32B units.
161 // Batch relay: one MTE2 + one MTE3 + one round-trip event per whole page
162 // instead of per-row relays (profiling iter2: mte2 49% + mte3 33% of the
163 // single-core time was per-row relay overhead).
164 __aicore__ inline void RelayDrain()
165 {

```

```

166 // UB->GM write is async on MTE3; before freeing the single-depth
167 // buffer for the next MTE2 reload, drain MTE3 so the next GM->UB copy
168 // cannot overwrite data MTE3 has not yet read out (hot id 216 pattern;
169 // canonical pairing per TPipe::WriteSpmBuffer: MTE3_MTE2 event).
170 TEventID evRet = GetTPipePtr()->FetchEventID(HardEvent::MTE3_MTE2);
171 SetFlag<HardEvent::MTE3_MTE2>(evRet);
172 WaitFlag<HardEvent::MTE3_MTE2>(evRet);
173 GetTPipePtr()->ReleaseEventID<HardEvent::MTE3_MTE2>(evRet);
174 }
175
176 // Page-batched relay: ONE MTE2 load of nElems contiguous GM elems into UB,
177 // then per-row 1D MTE3 writes (blockLenBytes each) to scattered dst rows
178 // (dstRowStrideElems apart). Rationale (iter19-24 evidence): a single big
179 // MTE3 1D write loses the tail row, and 2D MTE3 dstGap semantics corrupt
180 // columns -- only small 1D MTE3 writes are reliable; batching the MTE2
181 // load and the sync events cuts the per-row relay overhead.
182 __aice__ inline void RelayPage(GlobalTensor<bfloat16_t> &dstGm, uint64_t dstOff,
183                               GlobalTensor<bfloat16_t> &srcGm, uint64_t srcBase,
184                               uint32_t nElems, uint32_t rows,
185                               uint32_t rowElems, uint32_t dstRowStrideElems)
186 {
187     const uint16_t loadLen = static_cast<uint16_t>(nElems * sizeof(bfloat16_t));
188     const uint16_t lenB = static_cast<uint16_t>(rowElems * sizeof(bfloat16_t));
189     LocalTensor<bfloat16_t> qg = qgTQue_.AllocTensor<bfloat16_t>();
190     DataCopyPad<bfloat16_t>(qg, srcGm[srcBase],
191                            {1, loadLen, loadLen, loadLen, 0},
192                            {false, 0, 0, 0});
193     qgTQue_.EnQue(qg);
194     qg = qgTQue_.DeQue<bfloat16_t>();
195     TEventID ev = GetTPipePtr()->FetchEventID(HardEvent::MTE2_MTE3);
196     SetFlag<HardEvent::MTE2_MTE3>(ev);
197     WaitFlag<HardEvent::MTE2_MTE3>(ev);
198     GetTPipePtr()->ReleaseEventID<HardEvent::MTE2_MTE3>(ev);
199     for (uint32_t r = 0; r < rows; r++) {
200         // 1D MTE3 write per row: src UB offset r*rowElems (contiguous),
201         // dst row r at dstOff + r*dstRowStrideElems (scattered rows)
202         LocalTensor<bfloat16_t> srcView = qg[r * rowElems];
203         DataCopyPad<bfloat16_t>(dstGm[dstOff + static_cast<uint64_t>(r) * dstRowStrideElems], srcView, {1, lenB, lenB,
204 lenB, 0});
205     }
206     RelayDrain();
207     qgTQue_.FreeTensor(qg);
208 }
209
210 // Whole-page triple relay: ONE staging pass for the K nope + rope + V page
211 // slices of one KV page. Three MTE2 loads land in disjoint offsets of one
212 // UB buffer (nope | rope | V), then a single MTE2_MTE3 event pair + a
213 // single RelayDrain bracket all per-row 1D MTE3 writes. Replaces 3
214 // independent RelayPage calls (3 event pairs + 3 drains per page);
215 // profiling iter27/28 showed the scalar mte3-stall (event/drain wait) is
216 // the dominant cost, not the MTE2 bandwidth itself.
217 __aice__ inline void RelayPageKRV(GlobalTensor<bfloat16_t> &kvGm, uint64_t dst,
218                                   GlobalTensor<bfloat16_t> &vgGm, uint64_t dstV,
219                                   uint64_t src, uint64_t srcRope)
220 {
221     constexpr uint32_t NOPE_ELEMS = P_MAX * NOPE_DIM; // staging offset of rope
222     constexpr uint32_t V_OFF = P_MAX * (NOPE_DIM + ROPE_DIM); // staging offset of V
223     const uint16_t nopeB = static_cast<uint16_t>(P_ * NOPE_DIM * sizeof(bfloat16_t));
224     const uint16_t ropeB = static_cast<uint16_t>(P_ * ROPE_DIM * sizeof(bfloat16_t));
225     const uint16_t rowNB = static_cast<uint16_t>(NOPE_DIM * sizeof(bfloat16_t));
226     const uint16_t rowRB = static_cast<uint16_t>(ROPE_DIM * sizeof(bfloat16_t));
227     LocalTensor<bfloat16_t> qg = qgTQue_.AllocTensor<bfloat16_t>();
228     DataCopyPad<bfloat16_t>(qg, keyGm[src],
229                            {1, nopeB, nopeB, nopeB, 0}, {false, 0, 0, 0});
230     DataCopyPad<bfloat16_t>(qg[NOPE_ELEMS], kRopeGm[srcRope],
231                            {1, ropeB, ropeB, ropeB, 0}, {false, 0, 0, 0});
232     DataCopyPad<bfloat16_t>(qg[V_OFF], valueGm[src],
233                            {1, nopeB, nopeB, nopeB, 0}, {false, 0, 0, 0});
234     qgTQue_.EnQue(qg);
235     qg = qgTQue_.DeQue<bfloat16_t>();
236     TEventID ev = GetTPipePtr()->FetchEventID(HardEvent::MTE2_MTE3);
237     SetFlag<HardEvent::MTE2_MTE3>(ev);
238     WaitFlag<HardEvent::MTE2_MTE3>(ev);
239     GetTPipePtr()->ReleaseEventID<HardEvent::MTE2_MTE3>(ev);
240     for (uint32_t r = 0; r < P_; r++) {
241         DataCopyPad<bfloat16_t>(kvGm[dst + static_cast<uint64_t>(r) * DQK],
242                                qg[r * NOPE_DIM], {1, rowNB, rowNB, rowNB, 0});
243         DataCopyPad<bfloat16_t>(kvGm[dst + NOPE_DIM + static_cast<uint64_t>(r) * DQK],
244                                qg[NOPE_ELEMS + r * ROPE_DIM], {1, rowRB, rowRB, rowRB, 0});
245         DataCopyPad<bfloat16_t>(vgGm[dstV + static_cast<uint64_t>(r) * NOPE_DIM],
246                                qg[V_OFF + r * NOPE_DIM], {1, rowNB, rowNB, rowNB, 0});

```

```

246     }
247     RelayDrain();
248     qgTQue_.FreeTensor(qg);
249 }
250
251 // Gather selected KV pages into KVg [S2s, 576] (K nope+rope, for bmm1)
252 // and Vg [S2s, 512] (value nope cols, for bmm2 - value is a SEPARATE input
253 // tensor, it is NOT shared with key despite V==K dim sizes).
254 // Page ph = block_table[b, sparse_indices[b,0,s1,i]].
255 __aicore__ inline void GatherKV(uint32_t b, uint32_t s1,
256                                GlobalTensor<bfloat16_t> &kvGm, GlobalTensor<bfloat16_t> &vgGm)
257 {
258     // block_table row for this batch in ONE MTE2 load; per-page scalar
259     // GM reads ( GetValue on GlobalTensor ) each pay full GM latency in
260     // the scalar pipe (profiling iter27: aic_scalar mte2/mte3-stall heavy).
261     LocalTensor<int32_t> btLocal = btTQue_.AllocTensor<int32_t>();
262     DataCopy(btLocal, btGm_[static_cast<uint64_t>(b) * btStride_], btStride_);
263     btTQue_.EnQueue(btLocal);
264     btLocal = btTQue_.DeQueue<int32_t>();
265     for (uint32_t base = 0; base < K_; base += N_BATCH) {
266         uint32_t n = (K_ - base < N_BATCH) ? (K_ - base) : N_BATCH;
267         // indices layout [B, N2=1, S1, K]; TQue EnQueue/DeQueue gives the
268         // built-in MTE2->V sync that PipeBarrier cannot (hot id 182)
269         LocalTensor<int32_t> idxLocal = idxTQue_.AllocTensor<int32_t>();
270         DataCopy(idxLocal, idxGm_[(static_cast<uint64_t>(b) * S1_ + s1) * K_ + base], n);
271         idxTQue_.EnQueue(idxLocal);
272         idxLocal = idxTQue_.DeQueue<int32_t>();
273         for (uint32_t i = 0; i < n; i++) {
274             int32_t sel = idxLocal.GetValue(i);
275             int32_t ph = btLocal.GetValue(sel);
276             uint64_t src = static_cast<uint64_t>(ph) * P_ * NOPE_DIM;
277             uint64_t srcRope = static_cast<uint64_t>(ph) * P_ * ROPE_DIM;
278             uint64_t dst = static_cast<uint64_t>(base + i) * P_ * DQK;
279             uint64_t dstV = static_cast<uint64_t>(base + i) * P_ * NOPE_DIM;
280             RelayPageKRV(kvGm, dst, vgGm, dstV, src, srcRope);
281         }
282         idxTQue_.FreeTensor(idxLocal);
283     }
284     btTQue_.FreeTensor(btLocal);
285 }
286
287 // Joined Q rows: q-head h in [headOff, headOff + hg_):
288 // query[b, s1, h, 0..512] nope + query_rope[b, s1, h, 0..64]
289 // -> Qj workspace [hg_,576]. Rows of one head group are contiguous in both
290 // GM sources -> two whole-group page relays instead of 2*hg_ row relays
291 // (each row relay paid its own event pair + drain; profiling iter27 showed
292 // scalar mte3-stall dominated the gather phase).
293 __aicore__ inline void BuildJoinedQ(uint32_t b, uint32_t s1, uint32_t headOff,
294                                     GlobalTensor<bfloat16_t> &qjGm)
295 {
296     uint64_t qo = (static_cast<uint64_t>(b) * S1_ + s1) * N1_ * NOPE_DIM +
297                  static_cast<uint64_t>(headOff) * NOPE_DIM;
298     uint64_t ro = (static_cast<uint64_t>(b) * S1_ + s1) * N1_ * ROPE_DIM +
299                  static_cast<uint64_t>(headOff) * ROPE_DIM;
300     RelayPage(qjGm, 0, queryGm_, qo, hg_ * NOPE_DIM, hg_, NOPE_DIM, DQK);
301     RelayPage(qjGm, NOPE_DIM, qRopeGm_, ro, hg_ * ROPE_DIM, hg_, ROPE_DIM, DQK);
302 }
303
304 // softmax over S2s for the hg_ joined rows of this head-group task
305 __aicore__ inline void SoftmaxRows(uint32_t task, uint32_t subIdx)
306 {
307     (void)task;
308     uint32_t coreId = GetBlockIdx() / GetSubBlockNum();
309     GlobalTensor<float> scoreGm = scoreWsGm_[coreId * scoreSlotElems_];
310     GlobalTensor<bfloat16_t> pGm;
311     pGm.SetGlobalBuffer(reinterpret_cast<__gm__ bfloat16_t*>(
312         pWsBase_ + coreId * pSlotElems_ * sizeof(bfloat16_t)));
313
314     uint32_t rowBegin = 0, rowEnd = hg_;
315     (void)subIdx; // sub0 handles all rows serially (see Process())
316     for (uint32_t r = rowBegin; r < rowEnd; r += SOFTMAX_ROWS) {
317         uint32_t m = (rowEnd - r < SOFTMAX_ROWS) ? (rowEnd - r) : SOFTMAX_ROWS;
318         SoftmaxRowBatch(scoreGm, pGm, r, m);
319     }
320 }
321
322 __aicore__ inline void SoftmaxRowBatch(GlobalTensor<float> &scoreGm, GlobalTensor<bfloat16_t> &pGm,
323                                       uint32_t rowBegin, uint32_t m)
324 {
325     LocalTensor<float> scoreFLocal = scoreTQue_.AllocTensor<float>();
326     for (uint32_t i = 0; i < m; i++) {

```

```

327     DataCopy(scoreFLocal[i * S2s_], scoreGm[(rowBegin + i) * S2s_], S2s_);
328 }
329 scoreTQue_.EnQue(scoreFLocal);
330 scoreFLocal = scoreTQue_.DeQue<float>();
331 // scale in float, cast to half for SoftMax (bf16 src unsupported), then
332 // half -> float -> bf16 two-step relay (no direct half->bf16 Cast on
333 // dav_c220; bf16 dst only accepts float src) so bmm2 A/B are both bf16.
334 LocalTensor<half> pHalfLocal = pHalfTBuf_.Get<half>();
335 Muls(scoreFLocal, scoreFLocal, scale_, m * S2s_);
336 Cast(pHalfLocal, scoreFLocal, RoundMode::CAST_RINT, m * S2s_);
337 scoreTQue_.FreeTensor(scoreFLocal);
338
339 LocalTensor<float> sumLocal = sumTBuf_.Get<float>();
340 LocalTensor<float> maxLocal = maxTBuf_.Get<float>();
341 SoftMaxShapeInfo shapeInfo{m, S2s_, m, S2s_};
342 SoftMax<half, false>(pHalfLocal, sumLocal, maxLocal, pHalfLocal, softmaxTiling_, shapeInfo);
343 LocalTensor<float> pFloatLocal = pFloatTBuf_.Get<float>();
344 Cast(pFloatLocal, pHalfLocal, RoundMode::CAST_NONE, m * S2s_);
345 LocalTensor<bfloat16_t> pLocal = pTQue_.AllocTensor<bfloat16_t>();
346 // CAST_NONE float->bf16 is rejected by dav_c220 (assert compiles to a
347 // silent skip in release) leaving stale UB data in dst - must use an
348 // explicit rounding mode.
349 Cast(pLocal, pFloatLocal, RoundMode::CAST_RINT, m * S2s_);
350 pTQue_.EnQue(pLocal);
351 pLocal = pTQue_.DeQue<bfloat16_t>();
352 for (uint32_t i = 0; i < m; i++) {
353     DataCopy(pGm[(rowBegin + i) * S2s_], pLocal[i * S2s_], S2s_);
354 }
355 pTQue_.FreeTensor(pLocal);
356 }
357
358 TQue<TPosition::VECIN, 1> scoreTQue_;
359 TQue<TPosition::VECCALC, 1> pTQue_;
360 TBuf<TPosition::VECCALC> pHalfTBuf_;
361 TBuf<TPosition::VECCALC> pFloatTBuf_;
362 TBuf<TPosition::VECCALC> sumTBuf_;
363 TBuf<TPosition::VECCALC> maxTBuf_;
364 TQue<TPosition::VECIN, 1> idxTQue_;
365 TQue<TPosition::VECIN, 1> btTQue_;
366 TQue<TPosition::VECIN, 1> qgTQue_;
367 GlobalTensor<bfloat16_t> queryGm_, keyGm_, valueGm_, qRopeGm_, kRopeGm_, outGm_;
368 GlobalTensor<bfloat16_t> qjGm_;
369 GlobalTensor<int32_t> idxGm_, btGm_;
370 GlobalTensor<float> scoreWsGm_;
371 __gm__ uint8_t *pWsBase_ = nullptr;
372 __gm__ uint8_t *kvWsBase_ = nullptr;
373 __gm__ uint8_t *vgWsBase_ = nullptr;
374 __gm__ uint8_t *qjWsBase_ = nullptr;
375 SoftMaxTiling softmaxTiling_;
376 uint32_t B_ = 0, S1_ = 0, N1_ = 0, P_ = 0, K_ = 0, gSize_ = 0, hg_ = 0, nGroups_ = 0;
377 uint32_t S2s_ = 0, usedCoreNum_ = 0, nIdxSize_ = 0, totalTasks_ = 0;
378 float scale_ = 1.0f;
379 uint64_t scoreSlotElems_ = 0, pSlotElems_ = 0, kvSlotBytes_ = 0, vgSlotBytes_ = 0, qjSlotBytes_ = 0;
380 };
381
382 extern "C" __global__ __aicore__ void sparse_flash_attention(
383     GM_ADDR query, GM_ADDR key, GM_ADDR value, GM_ADDR sparse_indices,
384     GM_ADDR block_table, GM_ADDR actual_seq_lengths_query, GM_ADDR actual_seq_lengths_kv,
385     GM_ADDR query_rope, GM_ADDR key_rope, GM_ADDR attention_out,
386     GM_ADDR workspace, GM_ADDR tiling)
387 {
388     GET_TILING_DATA(tilingData, tiling);
389     // MIX AIC 1:2: N=AIC servers + 2N AIV; vector drives cube via KFC
390     KERNEL_TASK_TYPE_DEFAULT(KERNEL_TYPE_MIX_AIC_1_2);
391     uint32_t blk = static_cast<uint32_t>(GetBlockIdx());
392     uint32_t sub = static_cast<uint32_t>(GetSubBlockNum());
393     if (blk >= tilingData.usedCoreNum * sub)
394         return;
395     set_mask_norm();
396     SparseFlashAttentionKernel op;
397     op.btStride_ = tilingData.sparseBlkCount; // max blocks per batch == selects
398     REGIST_MATMUL_OBJ(&op.pipe_, GetSysWorkspacePtr(), op.bmm1, &tilingData.cubeTiling1,
399         op.bmm2, &tilingData.cubeTiling2);
400     op.Init(query, key, value, sparse_indices, block_table, actual_seq_lengths_query,
401         actual_seq_lengths_kv, query_rope, key_rope, attention_out, workspace, tilingData);
402     op.Process();
403 }

```